



Siemens
Industry
Online
Support

TIA Scripting Python

Verwendung von Python-Skripten zur Automatisierung von TIA Portal-Prozessen.

SIEMENS

Rechtliche Hinweise

Nutzung der Anwendungsbeispiele

In den Anwendungsbeispielen wird die Lösung von Automatisierungsaufgaben im Zusammenspiel mehrerer Komponenten in Form von Text, Grafiken und/oder Software-Bausteinen beispielhaft dargestellt. Die Anwendungsbeispiele sind ein kostenloser Service der Siemens AG und/oder einer Tochtergesellschaft der Siemens AG ("Siemens"). Sie sind unverbindlich und erheben keinen Anspruch auf Vollständigkeit und Funktionsfähigkeit hinsichtlich Konfiguration und Ausstattung. Die Anwendungsbeispiele stellen keine kundenspezifischen Lösungen dar, sondern bieten lediglich Hilfestellung bei typischen Aufgabenstellungen. **Die Anwendungsbeispiele unterliegen nicht den Standardtests und Qualitätsprüfungen eines kostenpflichtigen Produkts und können funktionale und Leistungsdefekte sowie andere Fehler und Sicherheitslücken enthalten. Sie sind verantwortlich für den ordnungsgemäßen und sicheren Betrieb der Produkte gemäß allen geltenden Vorschriften, einschließlich der Überprüfung und Anpassung des Anwendungsbeispiels für Ihr System, und stellen sicher, dass nur geschultes Personal es so verwendet, dass Sachschäden oder Verletzungen von Personen vermieden werden. Sie sind allein verantwortlich für jede produktive Nutzung.**

Sie erhalten von Siemens das nicht ausschließliche, nicht unterlizenzierbare und nicht übertragbare Recht, die Anwendungsbeispiele durch fachlich geschultes Personal zu nutzen. Jede Änderung an den Anwendungsbeispielen erfolgt auf Ihre Verantwortung. Die Weitergabe an Dritte oder Vervielfältigung der Anwendungsbeispiele oder von Auszügen daraus ist nur in Kombination mit Ihren eigenen Produkten gestattet. Jede weitere Verwendung der Anwendungsbeispiele ist ausdrücklich nicht gestattet und es werden keine weiteren Rechte gewährt. Sie dürfen die Anwendungsbeispiele in keiner anderen Weise verwenden, insbesondere ist jegliches direkte oder indirekte Training oder Verbesserungen von KI-Modellen ausdrücklich untersagt.

Haftungsausschluss

Siemens schließt seine Haftung, gleich aus welchem Rechtsgrund, insbesondere für die Verwendbarkeit, Verfügbarkeit, Vollständigkeit und Mangelfreiheit der Anwendungsbeispiele, sowie dazugehöriger Hinweise, Projektierungs- und Leistungsdaten und dadurch verursachte Schäden aus. Dies gilt nicht, soweit Siemens zwingend haftet, z.B. nach dem Produkthaftungsgesetz, in Fällen des Vorsatzes, der groben Fahrlässigkeit, wegen der schuldhaften Verletzung des Lebens, des Körpers oder der Gesundheit, bei Nichteinhaltung einer übernommenen Garantie, wegen des arglistigen Verschweigens eines Mangels oder wegen der schuldhaften Verletzung wesentlicher Vertragspflichten. Der Schadensersatzanspruch für die Verletzung wesentlicher Vertragspflichten ist jedoch auf den vertragstypischen, vorhersehbaren Schaden begrenzt, soweit nicht Vorsatz oder grobe Fahrlässigkeit vorliegen oder wegen der Verletzung des Lebens, des Körpers oder der Gesundheit gehaftet wird. Eine Änderung der Beweislast zu Ihrem Nachteil ist mit den vorstehenden Regelungen nicht verbunden. Von in diesem Zusammenhang bestehenden oder entstehenden Ansprüchen Dritter stellen Sie Siemens frei, soweit Siemens nicht gesetzlich zwingend haftet.

Durch Nutzung der Anwendungsbeispiele erkennen Sie an, dass Siemens über die beschriebene Haftungsregelung hinaus nicht für etwaige Schäden haftbar gemacht werden kann.

Weitere Hinweise

Siemens behält sich das Recht vor, Änderungen an den Anwendungsbeispielen jederzeit ohne Ankündigung durchzuführen und Ihnen die Lizenz jederzeit zu kündigen. Bei Abweichungen zwischen den Vorschlägen in den Anwendungsbeispielen und anderen Siemens Publikationen, wie z. B. Katalogen, hat der Inhalt der anderen Dokumentation Vorrang.

Ergänzend gelten die Siemens Nutzungsbedingungen (<https://www.siemens.com/global/en/general/terms-of-use>).

Cybersecurity-Hinweise

Siemens bietet Produkte und Lösungen mit Industrial Cybersecurity-Funktionen an, die den sicheren Betrieb von Anlagen, Systemen, Maschinen und Netzwerken unterstützen.

Um Anlagen, Systeme, Maschinen und Netzwerke gegen Cyber-Bedrohungen zu sichern, ist es erforderlich, ein ganzheitliches Industrial Cybersecurity-Konzept zu implementieren (und kontinuierlich aufrechtzuerhalten), das dem aktuellen Stand der Technik entspricht. Die Produkte und Lösungen von Siemens formen einen Bestandteil eines solchen Konzepts.

Die Kunden sind dafür verantwortlich, unbefugten Zugriff auf ihre Anlagen, Systeme, Maschinen und Netzwerke zu verhindern. Diese Systeme, Maschinen und Komponenten sollten nur mit dem Unternehmensnetzwerk oder dem Internet verbunden werden, wenn und soweit dies notwendig ist und nur wenn entsprechende Schutzmaßnahmen (z.B. Firewalls und/oder Netzwerksegmentierung) ergriffen wurden.

Weiterführende Informationen zu möglichen Schutzmaßnahmen im Bereich Industrial Cybersecurity finden Sie unter www.siemens.com/cybersecurity-industry.

Die Produkte und Lösungen von Siemens werden ständig weiterentwickelt, um sie noch sicherer zu machen. Siemens empfiehlt ausdrücklich, Produkt-Updates anzuwenden, sobald sie zur Verfügung stehen und immer nur die aktuellen Produktversionen zu verwenden. Die Verwendung veralteter oder nicht mehr unterstützter Versionen kann das Risiko von Cyber-Bedrohungen erhöhen.

Um stets über Produkt-Updates informiert zu sein, abonnieren Sie den Siemens Industrial Cybersecurity RSS Feed unter <https://www.siemens.com/cert>.

Inhaltsverzeichnis

1.	TIA Scripting Python	17
1.1.	TIA Portal Programmierschnittstelle	17
1.2.	Voraussetzungen	17
1.2.1.	Installierte Software	17
1.3.	Python-Umgebung	17
1.3.1.	Python-Installation	17
1.3.2.	Python-Schnellstart	18
1.4.	Verwaltung der Rechte von TIA Portal Openness	19
1.4.1.	Benutzer zur Siemens TIA Openness-Gruppe hinzufügen	19
1.4.2.	Openness-Sicherheitsdialog	19
1.5.	So verwenden Sie TIA Scripting Python	20
1.5.1.	TIA Scripting Python per Dateiimport verwenden	20
1.5.2.	TIA Scripting Python als Python-Paket verwenden	20
1.6.	Skripte	20
1.7.	Erstellen Sie eine ausführbare Datei aus Ihrem Skript	21
1.7.1.	Schritte zur Erstellung eines ausführbaren Programms	21
1.8.	Hierarchie	23
2.	Funktionsliste	24
2.1.	Devices - Hmi	24
2.1.1.	get_name()	24
2.1.2.	get_hmi_type()	24
2.1.3.	open_device_editor()	24
2.1.4.	compile_hardware()	24
2.1.5.	compile_software()	24
2.1.6.	upgrade_hardware(full_upgrade: bool)	24
2.1.7.	get_property(name: str)	25
2.1.8.	get_properties()	25
2.1.9.	set_property(name:str, value: str)	25
2.1.10.	get_identifier()	25
2.1.11.	get_hmi_tag_tables(folder_path: Optional[str] = None)	26
2.1.12.	get_screens(folder_path: Optional[str] = None)	26
2.1.13.	get_text_lists()	26
2.1.14.	get_scripts(folder_path: Optional[str] = None)	26
2.1.15.	get_alarms()	26
2.1.16.	get_alarm_classes()	26
2.1.17.	get_connections()	26
2.1.18.	get_cycles()	27

2.1.19.	get_graphic_lists()	27
2.1.20.	get_global_screen_elements()	27
2.1.21.	get_screen_overview()	27
2.1.22.	get_slide_in_screens(folder_path: Optional[str] = None)	27
2.1.23.	import_hmi_tags(import_root_directory: str, target_folder_path: Optional[str] = None)	27
2.1.24.	import_connections(import_root_directory: str)	28
2.1.25.	import_cycles(import_root_directory: str)	28
2.1.26.	import_scripts(import_root_directory: str, target_folder_path: Optional[str] = None)	28
2.1.27.	import_text_lists(import_root_directory: str)	28
2.1.28.	import_graphic_lists(import_root_directory: str)	28
2.1.29.	import_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	28
2.1.30.	import_popup_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	29
2.1.31.	import_template_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	29
2.1.32.	import_slidein_screens(import_root_directory: str)	29
2.1.33.	import_global_elements(import_file: str)	29
2.1.34.	import_screen_overview(import_file: str)	29
2.2.	Devices - Plc	31
2.2.1.	get_name()	31
2.2.2.	open_device_editor()	31
2.2.3.	get_online_state()	31
2.2.4.	go_offline()	31
2.2.5.	go_online(pc_interface_type: str, pc_interface_name: str, target_interface: str)	31
2.2.6.	download(pc_interface_type: str, pc_interface_name: str, target_interface: str)	31
2.2.7.	download_to_memory_card(download_directory: str, is_plcsim_advanced: Optional[bool] = None)	32
2.2.8.	get_plc_tag_tables(folder_path: Optional[str] = None)	32
2.2.9.	update_module_description()	32
2.2.10.	compile_hardware()	32
2.2.11.	compile_software()	32
2.2.12.	upgrade_hardware(full_upgrade: bool)	32
2.2.13.	compare_to_online()	33
2.2.14.	get_program_blocks(folder_path: Optional[str] = None)	33
2.2.15.	get_system_blocks()	33
2.2.16.	get_user_data_types(folder_path: Optional[str] = None)	33
2.2.17.	get_external_sources(folder_path: Optional[str] = None)	33
2.2.18.	get_force_tables()	34
2.2.19.	get_watch_tables(folder_path: Optional[str] = None)	34
2.2.20.	get_technology_objects(folder_path: Optional[str] = None)	34
2.2.21.	get_software_units()	34
2.2.22.	get_safety_administration()	34
2.2.23.	import_blocks(import_root_directory: str)	34

2.2.24.	import_plc_tags(import_root_directory: str, target_folder_path: Optional[str] = None)	34
2.2.25.	import_data_types(import_root_directory: str, target_folder_path: Optional[str] = None)	35
2.2.26.	import_technology_objects(import_root_directory: str, target_folder_path: Optional[str] = None)	35
2.2.27.	import_watch_tables(import_root_directory: str, target_folder_path: Optional[str] = None)	35
2.2.28.	create_software_unit(name: str)	35
2.2.29.	safety_print(print_file: str)	36
2.2.30.	get_property(name: str)	36
2.2.31.	get_properties()	36
2.2.32.	set_property(name:str, value: str)	36
2.2.33.	get_identifier()	36
2.3.	Enums	38
2.3.1.	PortalMode	38
2.3.2.	UmacUserMode	38
2.3.3.	ExportFormats	38
2.3.4.	ExportOptions	38
2.3.5.	CleanUpMode	38
2.3.6.	LibraryExportOptions	38
2.3.7.	HarmonizeOptions	38
2.3.8.	DependenciesMode	38
2.4.	Global - Global	39
2.4.1.	open_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)	39
2.4.2.	attach_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)	39
2.4.3.	open_attach_project(project_file_path: str, portal_mode: Optional[Enums.PortalMode] = None, server_project_view: Optional[bool] = None)	39
2.4.4.	get_installed_bundles()	40
2.4.5.	get_installed_products()	40
2.4.6.	set_umac_credentials(user_name: str, user_password: str, user_type: Enums.UmacUserMode)	40
2.4.7.	encrypt_umac_config(umac_file_path: str, secret: str, secret_env_name: str)	40
2.4.8.	set_logging(path: str, console: bool)	40
2.5.	Global - Portal	41
2.5.1.	get_process_id()	41
2.5.2.	open_project(project_file_path: str, server_project_view: Optional[bool] = None)	41
2.5.3.	open_project_with_copy(project_file_path: str, target_directory_path: str, delete_existing_project: bool)	41
2.5.4.	retrieve_archive(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)	41
2.5.5.	create_project(target_directory_path: str, project_name: str, delete_existing_project: bool)	42
2.5.6.	get_project()	42
2.5.7.	get_global_library_infos()	42
2.5.8.	get_global_library(library_name: str)	42
2.5.9.	open_global_library(library_path: str)	42
2.5.10.	open_global_library_with_copy(target_directory_path: str, library_path: str, delete_existing_project: bool)	43

2.5.11.	create_global_library(target_directory_path: str, library_name: str, delete_existing_project: bool)	43
2.5.12.	retrieve_archive_library(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)	43
2.5.13.	close_portal()	43
2.5.14.	show_online_finger_prints(mode: str, pc_interface: str, ip_address: str)	44
2.5.15.	get_project_servers(server: Optional[str] = None)	44
2.5.16.	detach()	44
2.5.17.	add_project_server(url: str, name: str)	44
2.6.	Global - Product	45
2.6.1.	get_name()	45
2.6.2.	get_release()	45
2.6.3.	get_version()	45
2.7.	Global - ProductBundle	46
2.7.1.	get_title()	46
2.7.2.	get_release()	46
2.7.3.	get_products()	46
2.8.	HMI Data - HmiAlarm	47
2.8.1.	get_name()	47
2.8.2.	get_property(name: str)	47
2.8.3.	get_properties()	47
2.8.4.	set_property(name:str, value: str)	47
2.8.5.	get_identifier()	47
2.9.	HMI Data - HmiAlarmClass	48
2.9.1.	get_name()	48
2.9.2.	get_property(name: str)	48
2.9.3.	get_properties()	48
2.9.4.	set_property(name:str, value: str)	48
2.9.5.	get_identifier()	48
2.10.	HMI Data - HmiConnection	49
2.10.1.	get_name()	49
2.10.2.	get_property(name: str)	49
2.10.3.	get_properties()	49
2.10.4.	set_property(name:str, value: str)	49
2.10.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	49
2.10.6.	get_identifier()	50
2.11.	HMI Data - HmiCycle	51
2.11.1.	get_name()	51
2.11.2.	get_property(name: str)	51
2.11.3.	get_properties()	51
2.11.4.	set_property(name:str, value: str)	51

2.11.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	51
2.11.6.	get_identifier()	52
2.12.	HMI Data - HmiGraphicList	53
2.12.1.	get_name()	53
2.12.2.	get_property(name: str).....	53
2.12.3.	get_properties()	53
2.12.4.	set_property(name:str, value: str)	53
2.12.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	53
2.12.6.	get_identifier()	54
2.13.	HMI Data - HmiScreen.....	55
2.13.1.	get_name()	55
2.13.2.	get_property(name: str).....	55
2.13.3.	get_properties()	55
2.13.4.	set_property(name:str, value: str)	55
2.13.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	55
2.13.6.	get_identifier()	56
2.14.	HMI Data - HmiScript	57
2.14.1.	get_name()	57
2.14.2.	get_property(name: str).....	57
2.14.3.	get_properties()	57
2.14.4.	set_property(name:str, value: str)	57
2.14.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	57
2.14.6.	get_identifier()	58
2.15.	HMI Data - HmiTag	59
2.15.1.	get_name()	59
2.15.2.	get_property(name: str).....	59
2.15.3.	get_properties()	59
2.15.4.	set_property(name:str, value: str)	59
2.15.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	59
2.15.6.	get_identifier()	60
2.16.	HMI Data - HmiTagTable	61
2.16.1.	get_name()	61
2.16.2.	get_property(name: str).....	61
2.16.3.	get_properties()	61
2.16.4.	set_property(name:str, value: str)	61

2.16.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	61
2.16.6.	get_identifier()	62
2.17.	HMI Data - HmiTextList	63
2.17.1.	get_name()	63
2.17.2.	get_property(name: str).....	63
2.17.3.	get_properties()	63
2.17.4.	set_property(name:str, value: str)	63
2.17.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	63
2.17.6.	get_identifier()	64
2.18.	Library - GlobalLibrary.....	65
2.18.1.	get_name()	65
2.18.2.	save()	65
2.18.3.	get_author()	65
2.18.4.	get_path().....	65
2.18.5.	is_modified()	65
2.18.6.	is_read_only()	65
2.18.7.	update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)	65
2.18.8.	update_project(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)	66
2.18.9.	harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)	66
2.18.10.	archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)	67
2.18.11.	get_property(name: str).....	67
2.18.12.	get_properties()	67
2.18.13.	set_property(name:str, value: str)	67
2.18.14.	clean_up(folder_path: Optional[str] = None).....	68
2.18.15.	get_types().....	68
2.18.16.	find_library_type(library_type_name: str)	68
2.18.17.	close_global_library()	68
2.18.18.	create_folder(folder_path: str)	68
2.18.19.	delete_folder(folder_path: str)	68
2.19.	Library - GlobalLibraryInfo	70
2.19.1.	get_name()	70
2.19.2.	get_property(name: str).....	70
2.19.3.	get_properties()	70
2.19.4.	set_property(name:str, value: str)	70
2.20.	Library - LibraryType	71
2.20.1.	get_name()	71
2.20.2.	get_author()	71

2.20.3.	get_guid()	71
2.20.4.	get_versions()	71
2.20.5.	find_version(version: str)	71
2.20.6.	get_property(name: str)	71
2.20.7.	get_properties()	71
2.20.8.	set_property(name:str, value: str)	72
2.20.9.	get_path_full()	72
2.20.10.	get_path()	72
2.20.11.	get_comment()	72
2.20.12.	get_identifier()	72
2.21.	Library - LibraryTypeFolder	73
2.21.1.	get_name()	73
2.21.2.	get_folders()	73
2.21.3.	get_types()	73
2.21.4.	find_library_type(library_type_name: str)	73
2.21.5.	find_folder(folder_name: str)	73
2.22.	Library - LibraryTypeVersion	74
2.22.1.	get_author()	74
2.22.2.	get_guid()	74
2.22.3.	get_version_number()	74
2.22.4.	get_modified_date()	74
2.22.5.	get_state()	74
2.22.6.	get_type_object()	74
2.22.7.	get_property(name: str)	74
2.22.8.	get_properties()	75
2.22.9.	set_property(name:str, value: str)	75
2.22.10.	find_instances(device_name: Optional[str] = None)	75
2.22.11.	instantiate(device_name: str, software_unit_name: Optional[str] = None, folder_path: Optional[str] = None)	75
2.22.12.	get_supported_export_format()	76
2.22.13.	export(target_directory_path: str, export_format: str, library_export_options: Enums.LibraryExportOptions, keep_folder_structure: Optional[bool] = None)	76
2.22.14.	edit(device_name: Optional[str] = None)	76
2.22.15.	release(dependencies_mode: Enums.DependenciesMode, version: str, author: str, comment: str)	76
2.22.16.	discard()	76
2.22.17.	set_as_default()	77
2.22.18.	get_comment()	77
2.22.19.	get_identifier()	77
2.23.	Library - ProjectLibrary	78
2.23.1.	get_type_folder()	78
2.23.2.	find_library_type(library_type_name: str)	78

2.23.3.	<code>clean_up(folder_path: Optional[str] = None)</code>	78
2.23.4.	<code>import_library_types(import_root_directory: str, device_name: Optional[str] = None, software_unit_name: Optional[str] = None, plc_folder_path: Optional[str] = None, library_folder_path: Optional[str] = None)</code>	78
2.23.5.	<code>update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)</code>	79
2.23.6.	<code>update_project(delete_mode: int, type_guids: Optional[List[str]] = None)</code>	79
2.23.7.	<code>harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)</code>	79
2.23.8.	<code>create_folder(folder_path: str)</code>	80
2.23.9.	<code>delete_folder(folder_path: str)</code>	80
2.24.	PLC Data - ExternalSource	81
2.24.1.	<code>get_name()</code>	81
2.24.2.	<code>get_property(name: str)</code>	81
2.24.3.	<code>block_gen()</code>	81
2.24.4.	<code>delete()</code>	81
2.24.5.	<code>get_properties()</code>	81
2.24.6.	<code>set_property(name:str, value: str)</code>	81
2.24.7.	<code>get_supported_export_format()</code>	82
2.24.8.	<code>get_identifier()</code>	82
2.25.	PLC Data - ForceTable	83
2.25.1.	<code>get_name()</code>	83
2.25.2.	<code>get_property(name: str)</code>	83
2.25.3.	<code>export(target_directory_path: str, export_options: Enums.ExportOptions, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)</code>	83
2.25.4.	<code>is_consistent()</code>	83
2.25.5.	<code>show_in_editor()</code>	83
2.25.6.	<code>get_properties()</code>	83
2.25.7.	<code>set_property(name:str, value: str)</code>	84
2.25.8.	<code>get_supported_export_format()</code>	84
2.25.9.	<code>get_path_full()</code>	84
2.25.10.	<code>get_path()</code>	84
2.25.11.	<code>get_fingerprints()</code>	84
2.25.12.	<code>get_identifier()</code>	84
2.26.	PLC Data - NamedValueType	86
2.26.1.	<code>get_name()</code>	86
2.26.2.	<code>get_namespace()</code>	86
2.26.3.	<code>export(target_directory_path: str, keep_folder_structure: Optional[bool] = None)</code>	86
2.27.	PLC Data - PlcTag	87
2.27.1.	<code>get_name()</code>	87
2.27.2.	<code>get_property(name: str)</code>	87
2.27.3.	<code>export(target_directory_path: str, export_options: Enums.ExportOptions, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)</code>	87

2.27.4.	delete()	87
2.27.5.	get_properties()	87
2.27.6.	set_property(name:str, value: str)	87
2.27.7.	get_supported_export_format()	88
2.27.8.	get_identifier()	88
2.28.	PLC Data - PlcTagTable	89
2.28.1.	get_name()	89
2.28.2.	get_property(name: str)	89
2.28.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	89
2.28.4.	get_plc_tags()	89
2.28.5.	get_user_constants()	89
2.28.6.	export_cross_references(target_directory_path: str, filter: int)	90
2.28.7.	show_in_editor()	90
2.28.8.	delete()	90
2.28.9.	get_properties()	90
2.28.10.	set_property(name:str, value: str)	90
2.28.11.	get_supported_export_format()	90
2.28.12.	get_path_full()	91
2.28.13.	get_path()	91
2.28.14.	get_fingerprints()	91
2.28.15.	get_identifier()	91
2.29.	PLC Data - ProgramBlock	92
2.29.1.	get_name()	92
2.29.2.	get_property(name: str)	92
2.29.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	92
2.29.4.	compile()	92
2.29.5.	is_consistent()	92
2.29.6.	export_cross_references(target_directory_path: str, filter: int)	93
2.29.7.	show_in_editor()	93
2.29.8.	get_type_version_guid()	93
2.29.9.	get_type_guid()	93
2.29.10.	delete()	93
2.29.11.	get_properties()	93
2.29.12.	get_path_full()	93
2.29.13.	get_path()	94
2.29.14.	set_property(name:str, value: str)	94
2.29.15.	get_supported_export_format()	94
2.29.16.	get_fingerprints()	94

2.29.17.	is_library_type()	94
2.29.18.	set_know_how_protection(password: str)	94
2.29.19.	remove_know_how_protection(password: str)	95
2.29.20.	get_identifier()	95
2.30.	PLC Data - SafetyAdministration	96
2.30.1.	is_logged_on()	96
2.30.2.	is_password_set()	96
2.30.3.	get_offline_serial_number()	96
2.30.4.	export_config(target_directory_path: str)	96
2.30.5.	import_config(import_root_directory: str)	96
2.31.	PLC Data - SoftwareUnit	97
2.31.1.	get_name()	97
2.31.2.	compile()	97
2.31.3.	export_configuration(export_file_path: str)	97
2.31.4.	import_configuration(import_file_path: str)	97
2.31.5.	get_plc_tag_tables()	97
2.31.6.	get_program_blocks()	97
2.31.7.	get_system_blocks()	97
2.31.8.	get_user_data_types()	98
2.31.9.	get_external_sources()	98
2.31.10.	get_named_value_types()	98
2.31.11.	import_blocks(import_root_directory: str)	98
2.31.12.	import_plc_tags(import_root_directory: str)	98
2.31.13.	import_data_types(import_root_directory: str)	98
2.31.14.	export_cross_references(target_directory_path: str, filter: int)	98
2.31.15.	get_property(name: str)	99
2.31.16.	get_properties()	99
2.31.17.	delete()	99
2.31.18.	set_property(name:str, value: str)	99
2.31.19.	get_identifier()	99
2.32.	PLC Data - SystemBlock	101
2.32.1.	get_name()	101
2.32.2.	get_property(name: str)	101
2.32.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	101
2.32.4.	compile()	101
2.32.5.	is_consistent()	101
2.32.6.	export_cross_references(target_directory_path: str, filter: int)	102
2.32.7.	show_in_editor()	102
2.32.8.	delete()	102

2.32.9.	get_properties()	102
2.32.10.	get_path_full()	102
2.32.11.	get_path()	102
2.32.12.	set_property(name:str, value: str)	102
2.32.13.	get_supported_export_format()	103
2.32.14.	get_fingerprints()	103
2.32.15.	get_identifier()	103
2.33.	PLC Data - TechnologyObject	104
2.33.1.	get_name()	104
2.33.2.	get_property(name: str)	104
2.33.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	104
2.33.4.	compile()	104
2.33.5.	is_consistent()	104
2.33.6.	delete()	105
2.33.7.	get_properties()	105
2.33.8.	get_path_full()	105
2.33.9.	get_path()	105
2.33.10.	set_property(name:str, value: str)	105
2.33.11.	get_supported_export_format()	105
2.33.12.	get_fingerprints()	105
2.33.13.	get_identifier()	106
2.34.	PLC Data - UserConstant	107
2.34.1.	get_name()	107
2.34.2.	get_property(name: str)	107
2.34.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	107
2.34.4.	delete()	107
2.34.5.	get_properties()	107
2.34.6.	set_property(name:str, value: str)	108
2.34.7.	get_supported_export_format()	108
2.34.8.	get_identifier()	108
2.35.	PLC Data - UserData Type	109
2.35.1.	get_name()	109
2.35.2.	get_property(name: str)	109
2.35.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	109
2.35.4.	compile()	109
2.35.5.	is_consistent()	109
2.35.6.	export_cross_references(target_directory_path: str, filter: int)	110
2.35.7.	get_type_version_guid()	110

2.35.8.	get_type_guid().....	110
2.35.9.	delete().....	110
2.35.10.	get_properties()	110
2.35.11.	get_path_full()	110
2.35.12.	get_path().....	110
2.35.13.	set_property(name:str, value: str)	111
2.35.14.	get_supported_export_format()	111
2.35.15.	get_fingerprints()	111
2.35.16.	is_library_type()	111
2.35.17.	get_identifier()	111
2.36.	PLC Data - WatchTable	112
2.36.1.	get_name()	112
2.36.2.	get_property(name: str).....	112
2.36.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	112
2.36.4.	is_consistent().....	112
2.36.5.	show_in_editor()	112
2.36.6.	delete().....	113
2.36.7.	get_properties()	113
2.36.8.	set_property(name:str, value: str)	113
2.36.9.	get_supported_export_format()	113
2.36.10.	get_path_full()	113
2.36.11.	get_path().....	113
2.36.12.	get_fingerprints()	113
2.36.13.	get_identifier()	114
2.37.	Project - Project	115
2.37.1.	get_portal().....	115
2.37.2.	save()	115
2.37.3.	close()	115
2.37.4.	set_simulation_support(value: bool)	115
2.37.5.	archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)	115
2.37.6.	save_as(target_directory_path: str, project_name: str)	115
2.37.7.	export_cax_data(export_file_path: str, log_file_path: str)	116
2.37.8.	import_cax_data(import_file_path: str, log_file_path: str)	116
2.37.9.	open_topology_editor()	116
2.37.10.	open_network_editor().....	116
2.37.11.	get_plcs().....	116
2.37.12.	get_hmis()	116
2.37.13.	get_application_tests()	117
2.37.14.	get_system_tests()	117

2.37.15.	get_rule_sets()	117
2.37.16.	get_project_library()	117
2.37.17.	web_block_generate()	117
2.37.18.	upgrade_hardware(full_upgrade: bool)	117
2.37.19.	sivarc_generate()	118
2.37.20.	update_module_description()	118
2.37.21.	set_virtual_plc_support(value: bool)	118
2.37.22.	import_umac_config(import_file_path: str)	118
2.37.23.	export_umac_config(export_file_path: str)	118
2.37.24.	import_password_policy(import_file_path: str)	118
2.37.25.	export_password_policy(export_file_path: str)	119
2.37.26.	delete()	119
2.37.27.	start_transaction(undo_text: str, dialog_text: str)	119
2.37.28.	end_transaction(rollback: Optional[bool] = None)	119
2.37.29.	update_transaction(dialog_text: str)	119
2.37.30.	get_property(name: str)	119
2.37.31.	get_properties()	120
2.37.32.	set_property(name:str, value: str)	120
2.37.33.	is_session()	120
2.37.34.	commit_and_close(commit_message: str)	120
2.37.35.	get_identifier()	120
2.38.	Project - ProjectServer	121
2.38.1.	get_host()	121
2.38.2.	get_port()	121
2.38.3.	get_server_name()	121
2.38.4.	print_info()	121
2.38.5.	get_property(name: str)	121
2.38.6.	get_properties()	121
2.38.7.	set_property(name:str, value: str)	121
2.38.8.	delete()	122
2.38.9.	add_project(project_path: str)	122
2.38.10.	create_local_session(server_project_path: str, session_directory_path: str, session_name: str, exclusive: Optional[bool] = None)	122
2.38.11.	get_server_project_paths(group: Optional[str] = None)	122
2.38.12.	delete_local_session(session_file_path: str, server_project_path: str)	123
2.39.	Test Suite - ApplicationTest	124
2.39.1.	get_name()	124
2.39.2.	get_property(name: str)	124
2.39.3.	export(target_directory_path: str)	124
2.39.4.	set_scope(plc_name: str, instance_name: Optional[str] = None, execution_mode: Optional[int] = None)	124

2.40.	Test Suite - RuleSet	125
2.40.1.	get_name()	125
2.40.2.	get_property(name: str).....	125
2.40.3.	export(target_directory_path: str)	125
2.41.	Test Suite - SystemTest	126
2.41.1.	get_name()	126
2.41.2.	export(target_directory_path: str)	126
2.41.3.	set_scope(opcua_server_address: str, opcua_server_interface_type: int, opcua_server_interface_folder_path: Optional[str] = None)	126
2.41.4.	get_property(name: str).....	126
2.41.5.	get_properties()	126
2.41.6.	set_property(name:str, value: str)	127
2.41.7.	get_identifier()	127
3.	Anhang	128
3.1.	Service und Support	128
3.2.	Links und Literatur.....	129
3.3.	Änderungsdokumentation	129

1. TIA Scripting Python

TIA Scripting Python ist ein Modul zum Schreiben von Python-Skripten für die Interaktion mit der TIA Portal Openness Schnittstelle. Es wurde entwickelt, um einfache Aufgaben in TIA Portal Projekten mit Hilfe einer einfachen Skriptsprache zu automatisieren und so die Notwendigkeit einer komplizierten Programmierung zu vermeiden. Es hilft bei der Vereinfachung von Aufgaben wie dem Importieren und Übersetzen von Programmbausteinen, dem Exportieren von Daten und dem Vergleichen von Projekten. Dieses Modul vereinfacht einige der grundlegenden Aufgaben von TIA Portal und erleichtert Benutzern ohne viel Erfahrung die Arbeit mit TIA Portal Openness. Darüber hinaus werden die installierten TIA Portal Versionen automatisch erkannt und verwendet.

1.1. TIA Portal Programmierschnittstelle

Die TIA Portal Openness API (Application Programming Interface) von TIA Portal ermöglicht es Ihnen, wiederkehrende Schritte in Ihren Projekten zu automatisieren. Dies ist sinnvoll, da manuelle Anpassungen in Projekten eine hohe Fehleranfälligkeit aufweisen können. Außerdem können Sie durch diese Automatisierung Zeit sparen und effizienter arbeiten.

1.2. Voraussetzungen

1.2.1. Installierte Software

Um dieses Programm ausführen zu können, muss die folgende Software installiert sein:

- TIA Portal V15.1 (oder neuere Version)
- TIA Portal Openness V15.1 (oder neuer)

Die Anforderungen für die genannten Programme sind in den jeweiligen Dokumentationen zu finden. Um mit TIA Scripting Python zu arbeiten, ist Python erforderlich.

- Python 3.12.X, 3.13.X oder 3.14.X
- eine beliebige Python-IDE, z. B. Visual Studio Code

1.3. Python-Umgebung

1.3.1. Python-Installation

Um zu überprüfen, ob Python auf einem Windows-PC installiert ist, suchen Sie in der Startleiste nach Python oder führen Sie folgenden Befehl in der Eingabeaufforderung (cmd.exe) aus:

```
python --version
```

Falls keine Python-Version installiert ist, können Sie diese unter <https://www.python.org/> herunterladen. Für die Nutzung wird Python Version 3.12.X, 3.13.X oder 3.14.X benötigt.

Hinweis

Damit pip-Befehle funktionieren, muss Python in der PATH-Umgebungsvariable Ihres Systems enthalten sein.

1. Beim Installieren von Python achten Sie darauf, die Option **"Add python.exe to PATH"** im Installer zu aktivieren.
2. Falls Python bereits installiert ist und nicht im PATH enthalten ist, können Sie dies manuell hinzufügen:
 - Suchen Sie den Ordner, in dem `python.exe` installiert ist (z.B. `C:\Users\IhrName\AppData\Local\Programs\Python\Python312\`).
 - Öffnen Sie das Startmenü, suchen Sie nach "Umgebungsvariablen" und wählen Sie **Systemumgebungsvariablen bearbeiten**.
 - Im Fenster "Systemeigenschaften" klicken Sie auf **Umgebungsvariablen...**
 - Unter **Systemvariablen** (oder **Benutzervariablen**) suchen und wählen Sie die Variable **Path** aus und klicken Sie auf **Bearbeiten**.
 - Klicken Sie auf **Neu** und fügen Sie den Pfad zum Ordner mit `python.exe` hinzu.
 - Klicken Sie auf **OK**, um alle Dialoge zu schließen und die Änderungen zu speichern.
3. Nach dem Aktualisieren des PATH starten Sie alle geöffneten Eingabeaufforderungen oder Terminals neu.

ACHTUNG

TIA Scripting Python benötigt **Python Version 3.12.X, 3.13.X oder 3.14.X**.

Sie müssen Python 3.12, 3.13 oder 3.14 verwenden (beliebige Patch-Version, z.B. 3.12.0, 3.13.1, 3.14.0, usw.).

Andere Versionen, einschließlich älterer wie 3.11.X oder neuerer über 3.14.X hinaus, werden nicht unterstützt und funktionieren nicht.

Falls bereits eine andere Python-Version installiert ist, müssen Sie trotzdem explizit eine der unterstützten Versionen (3.12.X, 3.13.X oder 3.14.X) installieren.

1.3.2. Python-Schnellstart

Python ist eine interpretierte Programmiersprache, was bedeutet, dass Entwickler Python-Dateien (.py) in einem Texteditor schreiben und diese Dateien dann zur Ausführung in den Python-Interpreter geben. Eine Python-Datei wird in der Befehlszeile wie folgt ausgeführt:

```
python helloworld.py
```

Wobei „helloworld.py“ der Name Ihrer Python-Datei ist.

1.4. Verwaltung der Rechte von TIA Portal Openness

1.4.1. Benutzer zur Siemens TIA Openness-Gruppe hinzufügen

Wenn Sie TIA Portal Openness auf dem PC installieren, wird automatisch die Benutzergruppe "Siemens TIA Openness" angelegt.

Wenn Sie mit Ihrer TIA Portal Openness Anwendung auf TIA Portal zugreifen, verifiziert TIA Portal, dass Sie Mitglied der Benutzergruppe "Siemens TIA Openness" sind, entweder direkt oder indirekt über eine andere Benutzergruppe. Wenn Sie Mitglied der Benutzergruppe "Siemens TIA Openness" sind, startet die Anwendung TIA Portal Openness und baut eine Verbindung zu TIA Portal auf.

Befolgen Sie die Anweisungen im [Openness-Handbuch](#) für detaillierte Anweisungen.

1.4.2. Openness-Sicherheitsdialog

Wenn Sie die Anwendung zum ersten Mal verwenden und sie sich über TIA Portal Openness mit TIA Portal verbindet, werden Sie von TIA Portal über einen Sicherheitsdialog aufgefordert, die Verbindung zu akzeptieren oder abzulehnen:

- Wenn Sie Ihre TIA Portal Openness-Anwendung nur einmalig mit TIA Portal verbinden möchten, klicken Sie in der Eingabeaufforderung auf "Ja". Wenn Ihre TIA Portal Openness Anwendung das nächste Mal versucht, sich mit TIA Portal zu verbinden, wird die Abfrage erneut angezeigt.
- Um einen Whitelist-Eintrag für Ihre TIA Portal Openness-Anwendung zu erstellen, gehen Sie folgendermaßen vor:
 - Klicken Sie in der Eingabeaufforderung auf "Ja, für alle", um ein Dialogfeld zur Benutzerkontensteuerung anzuzeigen.
 - Klicken Sie im Dialog Benutzerkontensteuerung auf "Ja", um Ihre Anwendung zur Whitelist von TIA Portal in der Windows-Registry hinzuzufügen.

Weitere Informationen finden Sie im [Openness-Handbuch](#).

1.5. So verwenden Sie TIA Scripting Python

Es gibt zwei Möglichkeiten, TIA Scripting Python zu verwenden.

1.5.1. TIA Scripting Python per Dateiimport verwenden

Entpacken Sie TiaScriptingPython_x.x.x.zip in einen Ordner auf Ihrer Festplatte. Um diesen Ordner während der Ausführung von Skripten bekannt zu machen, setzen Sie eine globale Umgebungsvariable „TIA_SCRIPTING“, wobei der Wert der Pfad der zuvor entpackten TIA Scripting Python Binaries ist. Auf diese Weise wird „TIA_SCRIPTING“ von Python-Skripten verwendet, um das Python-Modul des TIA Scripting Python zu finden.

1.5.2. TIA Scripting Python als Python-Paket verwenden

Sie können TIA Scripting Python als Python-Paket mit `pip` (Pythons Paketmanager) installieren. Gehen Sie wie folgt vor:

Installationsschritte:

1. Öffnen Sie eine Eingabeaufforderung oder ein Terminal.
2. Navigieren Sie zu dem Verzeichnis, in dem sich die Datei `siemens_tia_scripting-x.x.x-cp31x-cp31x-win_amd64.whl` befindet:

```
cd C:\Path\To\Your\TIA_Scripting_Python\binaries\
```

(Ersetzen Sie `C:\Path\To\Your\TIA_Scripting_Python\binaries\` durch den tatsächlichen Pfad zu Ihrem TIA Scripting Python Binaries-Ordner.)

3. Installieren Sie das Paket mit `pip`:

```
pip install siemens_tia_scripting-x.x.x-cp31x-cp31x-win_amd64.whl
```

- Ersetzen Sie `x.x.x` durch die tatsächliche Versionsnummer Ihres TIA Scripting Python-Pakets (z.B. `1.0.0`).
- Der Teil `cp31x-cp31x-win_amd64` kann je nach Ihrer Python-Version (z.B.: 3.12, 3.13 and 3.14) und Systemarchitektur variieren, verwenden Sie also den genauen Dateinamen, den Sie haben.

Was passiert nach der Installation? Dieser Befehl installiert das TIA Scripting Python-Paket in das `site-packages`-Verzeichnis Ihrer aktuellen Python-Umgebung. Dadurch werden seine Module und Funktionalitäten für den Import und die Verwendung in Ihren Python-Skripten verfügbar.

Intellisense-Unterstützung

Intellisense-Unterstützung kann nur verwendet werden, wenn Sie TIA Scripting Python als Python-Paket installieren. Der Pylance Language Server (Python Intellisense) kann vollständige Intellisense-Unterstützung bieten. Pylance ist standardmäßig im Visual Studio Code Python Plugin und Microsoft Visual Studio Python enthalten. Andere Language Server wie „Jedi“ sind derzeit ungetestet.

```
portal = ts.attach_to_portal(show_gui = portal_mode_ui, version = version)

(function) def attach_to_portal(
    show_gui: Any,
    version: Any
) -> TiaPortal

Attach to running TIA Portal instance showgui can be True or False version consists of major.minor e.g. "17.0"
*Returns* -> TiaPortal TIA Portal instance

parameters type description
show_gui bool Set if gui should be displayed or not, by default False
version string consists of major.minor e.g. 17.0, by default selects latest installed Tia Portal
portal = siemens_tiaportal_scripting.attach_to_portal(show_gui = True, version = "18.0")
```

1.6. Skripte

Der Ordner „Scripts“ enthält Beispielskripte, die TIA Scripting Python verwenden, um mit TIA Portal zu interagieren.

Skript	Beschreibung
attacher.py	Verbindet sich mit einer laufenden TIA Portal Instanz (mit Benutzeroberfläche), ruft das geöffnete Projekt ab, gibt die Namen aller verfügbaren PLCs aus und öffnet den Geräteeditor der ersten PLC.
export_hmi.py	Exportiert HMI-bezogene Daten aus dem TIA Portal Projekt.
export_library.py	Exportiert Bibliothekselemente aus dem TIA Portal Projekt.
export_plc.py	Listet alle Softwareelemente einer PLC auf und exportiert sie in den konfigurierten Ordner.
importer.py	Öffnet ein TIA Portal Projekt und importiert alle exportierten PLC-Elemente (Datentypen, Technologieobjekte, Programmbausteine, PLC-Variablen, Beobachtungstabellen) aus einem angegebenen Verzeichnis in jede PLC im Projekt.
get_products.py	Listet alle installierten Siemens TIA Portal Produktpakete und deren Produkte auf, einschließlich Name, Release und Version. Listet auch alle installierten Produkte einzeln auf.

1.7. Erstellen Sie eine ausführbare Datei aus Ihrem Skript

Sie können Ihr Python-Skript mit PyInstaller in ein eigenständiges Windows-Executable (.exe) umwandeln. Dies bietet mehrere Vorteile:

- **Keine Python-Installation für Endbenutzer erforderlich:** Das ausführbare Programm kann auf jedem kompatiblen Windows-System ausgeführt werden, auch wenn Python nicht installiert ist.
- **Vereinfachte Verteilung:** Sie können eine einzelne .exe-Datei teilen, anstatt von Benutzern zu verlangen, eine Python-Umgebung und Abhängigkeiten einzurichten.
- **Schutz des Quellcodes:** Der Python-Quellcode ist gebündelt und nicht direkt zugänglich.

1.7.1. Schritte zur Erstellung eines ausführbaren Programms

1. **PyInstaller installieren** Öffnen Sie eine Eingabeaufforderung und führen Sie aus:

```
pip install pyinstaller
```

2. **Ihr Skript vorbereiten** Stellen Sie sicher, dass Ihr Skript (z.B. `demo.py`) bereit ist und sich in dem Ordner befindet, in dem Sie das ausführbare Programm erstellen möchten. Ersetzen Sie `demo.py` durch den Namen Ihres eigenen Skripts.

3. **PyInstaller ausführen** Führen Sie im selben Ordner wie Ihr Skript Folgendes aus:

```
pyinstaller --onefile demo.py
```

- `--onefile` weist PyInstaller an, alles in eine einzige .exe-Datei zu bündeln.
- `demo.py` ist Ihr zu konvertierendes Skript. Ersetzen Sie es durch den tatsächlichen Namen Ihres Skripts.
- Das resultierende ausführbare Programm wird im Unterordner `dist` abgelegt.

(Optional: Verwenden Sie die Option `-p`, um zusätzliche Pfade für Abhängigkeiten hinzuzufügen, z.B. `-p D:\PyLibs\`, wenn Sie benutzerdefinierte Bibliotheken haben.)

4. **Ihr ausführbares Programm finden** Nach Abschluss des Vorgangs suchen Sie im `dist`-Ordner nach Ihrer neuen .exe-Datei (z.B. `dist\demo.exe`).

Beim Erstellen eines Python-Executables ist es wichtig sicherzustellen, dass lokale Binaries (wie `siemens_tia_scripting.pyd`) anstelle von global installierten Paketen verwendet werden. Die folgenden Anweisungen führen Sie durch den Prozess, dies mit **PyInstaller** zu erreichen.

Um sicherzustellen, dass Ihr Python-Skript lokale Binaries lädt, fügen Sie den folgenden Code ein. Dieser findet die mit dem Executable gebündelte .pyd-Datei dynamisch:

```
import importlib.util
import os
import sys
```

```
# Die Datei vom Speicherort abrufen, der von PyInstaller extrahiert wurde
def get_file_path(file_name):
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, file_name)
    return os.path.join(os.path.abspath("."), file_name)

# Dieses spezifische Paket importieren, globale Installationen umgehen
spec = importlib.util.spec_from_file_location("siemens_tia_scripting",
get_file_path("siemens_tia_scripting.pyd"))
ts = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ts)
```

`get_file_path()`: Diese Funktion stellt sicher, dass die korrekte lokale Version der Binärdatei verwendet wird, insbesondere wenn das Skript in eine ausführbare Datei verpackt ist.

Der Code lädt dynamisch die `.pyd`-Binärdatei und stellt sicher, dass lokale Binärdateien verwendet werden, wobei alle global installierten Pakete umgangen werden.

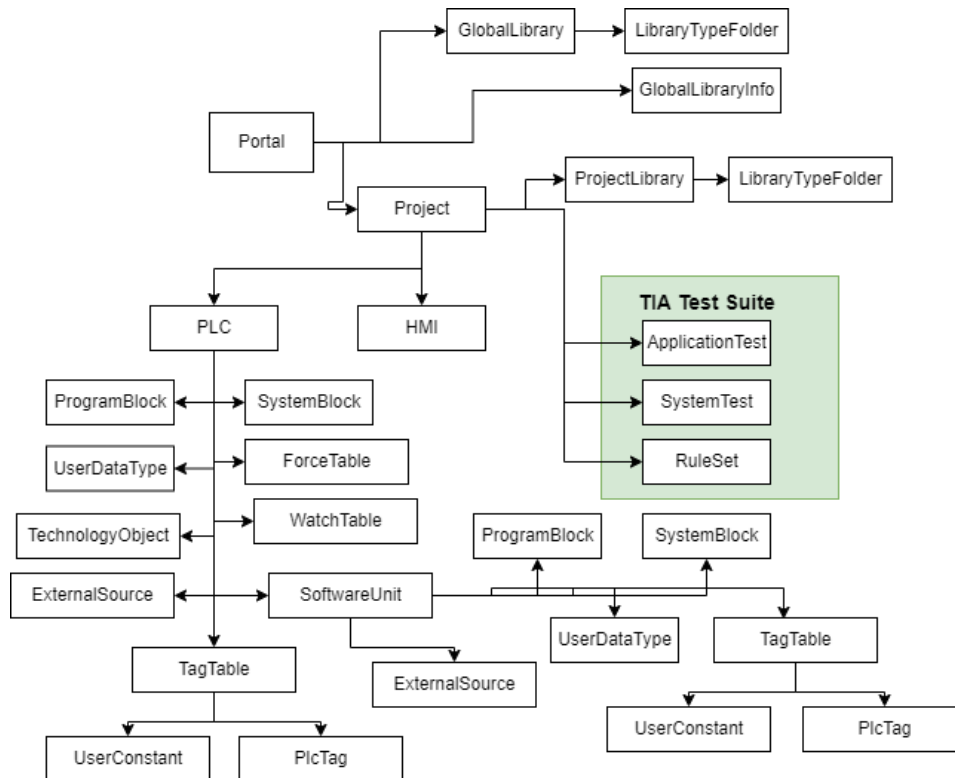
Sie können den folgenden PyInstaller-Befehl verwenden, um die ausführbare Datei direkt von der Kommandozeile aus zu erstellen:

```
pyinstaller -F /mytiascript.py --workpath /temp -n MyTiaExecutable --clean --distpath ./output
--add-data "../dep/:"
```

- **-F**: Erzeugt eine einzige eigenständige ausführbare Datei.
- **/mytiascript.py**: Der Pfad zu Ihrem Python-Skript, das kompiliert werden muss.
- **-Arbeitspfad /temp**: Gibt das temporäre Arbeitsverzeichnis während des Build-Prozesses an (in diesem Fall `../temp`).
- **-n MyTiaExecutable**: Gibt den Namen der resultierenden ausführbaren Datei an.
- **--clean**: Säubert alle temporären Dateien von früheren Builds, bevor es losgeht.
- **--distpath ./output**: Der Pfad, in dem die endgültige ausführbare Datei gespeichert wird (in diesem Fall `./output`).
- **--add-data "../dep/:"**: Fügt zusätzliche Daten (wie `.pyd`-Binärdateien) hinzu, die von der ausführbaren Datei benötigt werden (in diesem Fall der Inhalt des Verzeichnisses `../dep/`).

1.8. Hierarchie

Die Übersicht zeigt die verfügbaren Modelle und deren Abhängigkeiten zueinander. Der Zugriff auf einen Programmbaustein von einer PLC aus setzt voraus, dass eine TIA Portal-Verbindung hergestellt, ein Projekt geöffnet und ein Steuerungsobjekt ausgewählt wurde.



2. Funktionsliste

Die ersten Zeilen eines jeden Skripts sollten TIA Scripting Python importieren, wobei die zuvor definierte Systemumgebungsvariable dafür verwendet wird.

```
sys.path.append(os.getenv('TIA_SCRIPTING'))
import siemens_tia_scripting
```

Wenn der Import erfolgreich war, können die entsprechenden Methoden von TIA Scripting Python verwendet werden.

2.1. Devices - Hmi

Die Klasse repräsentiert HMI-Geräte (Human Machine Interface) in TIA Portal. Sie ermöglicht die Steuerung und Interaktion mit dem HMI, einschließlich Kompilierungs- und Aktualisierungsaufgaben.

2.1.1. get_name()

Abrufen des Namens des HMI

Returns → `str` Name des HMI

```
hmi_name = hmi.get_name()
```

2.1.2. get_hmi_type()

Abrufen des HMI-Typ

Returns → `str` Typ des HMI

```
hmi_type = hmi.get_hmi_type()
```

2.1.3. open_device_editor()

Öffnen des Editors des HMI-Gerätes in TIA Portal

```
hmi.open_device_editor()
```

2.1.4. compile_hardware()

Übersetzen der Hardware des HMI

Returns → `bool` Ergebnis der Übersetzung

Gibt `true` zurück, wenn die Übersetzung Fehler aufweist, andernfalls `false`.

```
result = hmi.compile_hardware()
```

2.1.5. compile_software()

Übersetzen der Software des HMI

Returns → `bool` Ergebnis der Übersetzung

Gibt `true` zurück, wenn die Übersetzung Fehler aufweist, andernfalls `false`.

```
result = hmi.compile_software()
```

2.1.6. upgrade_hardware(full_upgrade: bool)

Aktualisieren der Hardware des HMI

Wenn `full_upgrade` auf `True` gesetzt ist, werden alle Geräte auf die neueste verfügbare Bestellnummer (Gerätetyp) und Firmware geändert:

von CPU1511F 6ES7 511-1FK00-0AB0 V1.7 zu 6ES7 511-1FK00-0AB0 V1.8.

Mit vollständigem Upgrade: von CPU1511F 6ES7 511-1FK00-0AB0 V1.7 zu 6ES7 511-1FL03-0AB0 V3.0.

Parameter	Typ	Beschreibung
full_upgrade	bool	Legt fest, ob ein vollständiges Upgrade durchgeführt werden soll (neueste Bestellnummer und Firmware-Version)

```
hmi.upgrade_hardware(full_upgrade = True)
```

2.1.7. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.1.8. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.1.9. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.1.10. get_identifier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.1.11. get_hmi_tag_tables(folder_path: Optional[str] = None)

Abrufen der Liste der HMI Variablentabellen

Returns → `List[HmiTagTable]` Liste der HMI Variablentabellen

Parameter	Typ	Beschreibung
folder_path	str	Der Gruppenpfad des Elements

```
hmi_tag_tables = hmi.get_hmi_tag_tables(folder_path = "group1/group2")
```

2.1.12. get_screens(folder_path: Optional[str] = None)

Abrufen der Liste der HMI Bilder

Returns → `List[HmiScreen]` Liste der HMI Bilder

Parameter	Typ	Beschreibung
folder_path	str	Der Gruppenpfad des Elements

```
hmi_screens = hmi.get_screens(folder_path = "group1/group2")
```

2.1.13. get_text_lists()

Abrufen der Liste der HMI Textlisten

Returns → `List[HmiTextList]` Liste der HMI Textlisten

<pre>hmi_lists = hmi.get_text_lists()</pre>

2.1.14. get_scripts(folder_path: Optional[str] = None)

Abrufen der Liste der HMI-Skripte (VB Comfort), (JavaScript Unified)

Returns → `List[HmiScript]` Liste der HMI-Skripte

Parameter	Typ	Beschreibung
folder_path	str	Der Gruppenpfad des Elements

```
hmi_scripts = hmi.get_scripts(folder_path = "gruppe1/gruppe2")
```

2.1.15. get_alarms()

Abrufen der Liste der HMI-Alarme

Returns → `List[HmiAlarm]` Liste der HMI-Alarme

<pre>hmi_alarms = hmi.get_alarms()</pre>
--

2.1.16. get_alarm_classes()

Abrufen der Liste der HMI-Alarmklassen

Returns → `List[HmiAlarmClass]` Liste der HMI-Alarmklassen

<pre>hmi_alarms = hmi.get_alarms_classes()</pre>
--

2.1.17. get_connections()

Abrufen der Liste der HMI-Verbindungen

Returns → `List[HmiAlarm]` Liste der HMI-Verbindungen

```
connections = hmi.get_connections()
```

2.1.18. **get_cycles()**

Abrufen der Liste der HMI-Zyklen

Returns → `List[HmiCycle]` Liste der HMI-Zyklen

```
cycles = hmi.get_cycles()
```

2.1.19. **get_graphic_lists()**

Abrufen der Liste der HMI-Grafiklisten

Returns → `List[HmiGraphicList]` Liste der HMI-Grafiklisten

```
graph_lists = hmi.get_graphic_lists()
```

2.1.20. **get_global_screen_elements()**

Abrufen der globalen Bildelemente des HMI

Returns → `HmiScreen` Globale Bildelemente des HMI

```
global_elements = hmi.get_global_screen_elements()
```

2.1.21. **get_screen_overview()**

Abrufen der Bildübersicht des HMI

Returns → `HmiScreen` Bildübersicht des HMI

```
screen_overview = hmi.get_screen_overview()
```

2.1.22. **get_slide_in_screens(folder_path: Optional[str] = None)**

Abrufen der Liste der HMI-SlideIn-Bilder

Returns → `List[HmiScreen]` Liste der HMI-SlideIn-Bilder

Parameter	Typ	Beschreibung
folder_path	str	Der Gruppenpfad des Elements

```
hmi_slide_in_screens = hmi.get_slide_in_screens(folder_path = "gruppe1/gruppe2")
```

2.1.23. **import_hmi_tags(import_root_directory: str, target_folder_path: Optional[str] = None)**

Importieren der Variablen aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners
target_folder_path	str	Optional: Der Zielbasis-Pfad innerhalb der HMI-Gruppenstruktur, in dem die importierte Ordnerhierarchie wiederhergestellt wird

```
hmi.import_hmi_tags(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Tags")
```

2.1.24. import_connections(import_root_directory: str)

Importieren der Verbindungen aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners

```
hmi.import_hmi_tags(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Connections")
```

2.1.25. import_cycles(import_root_directory: str)

Importieren der Zyklen aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners

```
hmi.import_cycles(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Cycles")
```

2.1.26. import_scripts(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der Skripte aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners
target_folder_path	str	Optional: Der Zielbasis-Pfad innerhalb der HMI-Gruppenstruktur, in dem die importierte Ordnerhierarchie wiederhergestellt wird

```
hmi.import_scripts(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Scripts")
```

2.1.27. import_text_lists(import_root_directory: str)

Importieren der Textlisten aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners

```
hmi.import_text_lists(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Text lists")
```

2.1.28. import_graphic_lists(import_root_directory: str)

Importieren der Grafiklisten aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners

```
hmi.import_graphic_lists(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Graphics")
```

2.1.29. import_screens(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der Bilder aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners
target_folder_path	str	Optional: Der Zielbasis-Pfad innerhalb der HMI-Gruppenstruktur, in dem die importierte

Parameter	Typ	Beschreibung
		Ordnerhierarchie wiederhergestellt wird

```
hmi.import_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Screens")
```

2.1.30. **import_popup_screens(import_root_directory: str, target_folder_path: Optional[str] = None)**

Importieren der Popup-Bilder aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners
target_folder_path	str	Optional: Der Zielbasis-Pfad innerhalb der HMI-Gruppenstruktur, in dem die importierte Ordnerhierarchie wiederhergestellt wird

```
hmi.import_popup_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\PopUpScreens")
```

2.1.31. **import_template_screens(import_root_directory: str, target_folder_path: Optional[str] = None)**

Importieren der Template-Bilder aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners
target_folder_path	str	Optional: Der Zielbasis-Pfad innerhalb der HMI-Gruppenstruktur, in dem die importierte Ordnerhierarchie wiederhergestellt wird

```
hmi.import_template_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\TemplateScreens")
```

2.1.32. **import_slidein_screens(import_root_directory: str)**

Importieren der SlideIn-Bilder aus einem Verzeichnis in das HMI

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Import-Ordners

```
hmi.import_slidein_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\SlideinScreens")
```

2.1.33. **import_global_elements(import_file: str)**

Importieren der globale Elemente-Bilder in das HMI

Parameter	Typ	Beschreibung
import_file	str	Importdatei für globale Elemente

```
hmi.import_global_elements(import_file = "C:\\ws\\importfolder\\HMI_1\\GlobalElements.xml")
```

2.1.34. **import_screen_overview(import_file: str)**

Importieren der Bildübersicht in das HMI

Parameter	Typ	Beschreibung
import_file	str	Importdatei für die Bildübersicht

```
hmi.import_screen_overview(import_file = "C:\\ws\\importfolder\\HMI_1\\ScreenOverview.xml")
```

2.2. Devices - Plc

Dies stellt eine PLC (Programmable Logical Controller) dar. Sie ermöglicht die Steuerung und Interaktion mit der PLC, einschließlich des Ladens, Vergleich von Online- und Offline-Zuständen und Abruf von Informationen über PLC-Elemente.

2.2.1. get_name()

Abrufen des Namens der PLC

Returns → `str` Name der PLC

```
plc_name = plc.get_name()
```

2.2.2. open_device_editor()

Öffnen des Editor des PLC-Gerätes in TIA Portal

```
plc.open_device_editor()
```

2.2.3. get_online_state()

Ermitteln des aktuellen Online-Status der PLC

Returns → `str` Online-Status

```
plc.get_online_state()
```

2.2.4. go_offline()

Abbauen der Verbindung mit der PLC

```
plc.go_offline()
```

2.2.5. go_online(pc_interface_type: str, pc_interface_name: str, target_interface: str)

Aufbauen der Verbindung mit der PLC

Returns → `str` Online-Status

Parameter	Typ	Beschreibung
pc_interface_type	str	Konfigurationsmodus
pc_interface_name	str	PC-Schnittstellename
target_interface	str	API_V21: IP Adresse oder die Zielschnittstelle, wenn version unter v21 dann Zielschnittstelle

```
plc.go_online(pc_interface_type = "PN/IE", pc_interface_name = "PLCSIM", target_interface = "1X1")
```

2.2.6. download(pc_interface_type: str, pc_interface_name: str, target_interface: str)

Download auf die PLC

Returns → `str` Ergebnis des Downloads

Parameter	Typ	Beschreibung
pc_interface_type	str	Konfigurationsmodus
pc_interface_name	str	PC-Schnittstellename
target_interface	str	Zielschnittstelle


```
result = plc.download(pc_interface_type = "PN/IE", pc_interface_name = "PLCSIM",
target_interface = "1 X1")
```

2.2.7. download_to_memory_card(download_directory: str, is_plcsim_advanced: Optional[bool] = None)

Downloaden der PLC-Konfiguration auf eine Speicherkarte

Returns → `str` Ergebnis des Downloads

Parameter	Typ	Beschreibung
download_directory	str	Download-Verzeichnis
is_plcsim_advanced	bool	True: PLCSIM Advanced Download ist konfiguriert, standardmäßig False für den CPU-Download

```
result = plc.download_to_memory_card(download_directory = "E:\\", is_plcsim_advanced = True)
```

2.2.8. get_plc_tag_tables(folder_path: Optional[str] = None)

Abrufen der Liste der PLC-Variablen tabellen

Returns → `List[PlcTagTable]` Liste der PLC-Variablen tabellen

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
plc_tag_tables = plc.get_plc_tag_tables(folder_path = "group1/group2")
```

2.2.9. update_module_description()

Aktualisieren der Modulbeschreibung der PLC

Returns → `bool` Ergebnis der Aktualisierung

```
result = plc.update_module_description()
```

2.2.10. compile_hardware()

Übersetzen der Hardware der PLC

Returns → `bool` Ergebnis der Übersetzung

Gibt `true` zurück, wenn die Übersetzung Fehler aufweist, andernfalls `false`.

```
result = plc.compile_hardware()
```

2.2.11. compile_software()

Übersetzen der Software der PLC

Returns → `bool` Ergebnis der Übersetzung

Gibt `true` zurück, wenn die Übersetzung Fehler aufweist, andernfalls `false`.

```
result = plc.compile_software()
```

2.2.12. upgrade_hardware(full_upgrade: bool)

Aktualisieren der Hardware der PLC

Wenn `full_upgrade` auf `True` gesetzt ist, werden alle Geräte auf die neueste verfügbare Bestellnummer (Gerätetyp) und Firmware umgestellt:

von CPU1511F 6ES7 511-1FK00-0AB0 **V1.7** zu 6ES7 511-1FK00-0AB0 **V1.8**.

Mit vollständigem Upgrade: von CPU1511F 6ES7 511-1FK00-0AB0 **V1.7** zu 6ES7 511-1FL03-0AB0 **V3.0**.

Parameter	Typ	Beschreibung
full_upgrade	bool	Legt fest, ob ein vollständiges Upgrade durchgeführt werden soll (neueste Bestellnummer und Firmware-Version)

```
plc.upgrade_hardware(full_upgrade = True)
```

2.2.13. compare_to_online()

Vergleichen des aktuellen PLC-Zustands mit dem Online-Zustand

Returns → `bool` Ergebnis des Vergleichs

Gibt `false` zurück, wenn die Ordner identisch sind, andernfalls `true`.

```
result = plc.compare_to_online()
```

2.2.14. get_program_blocks(folder_path: Optional[str] = None)

Abrufen der Liste der Programmbausteine

Returns → `List[ProgramBlock]` Liste der Programmbausteine

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
blocks = plc.get_program_blocks(folder_path = "group1/group2")
```

2.2.15. get_system_blocks()

Abrufen der Liste der Systembausteine

Returns → `List[SystemBlock]` Liste der Systembausteine

```
blocks = plc.get_system_blocks()
```

2.2.16. get_user_data_types(folder_path: Optional[str] = None)

Abrufen der Liste der PLC-Datentypen

Returns → `List[UserDataTypes]` Liste der PLC-Datentypen

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
udts = plc.get_user_data_types(folder_path = "group1/group2")
```

2.2.17. get_external_sources(folder_path: Optional[str] = None)

Abrufen der Liste der externen Quellen

Returns → `List[ExternalSource]` Liste der externen Quellen der PLC

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
ext_sources = plc.get_external_sources()
```

2.2.18. get_force_tables()

Abrufen der Liste der Forcetabellen

Returns → `List[ForceTable]` Liste der Forcetabellen der PLC

```
tables = plc.get_force_tables()
```

2.2.19. get_watch_tables(folder_path: Optional[str] = None)

Abrufen der Liste der Beobachtungstabellen

Returns → `List[WatchTable]` Liste der Beobachtungstabellen der PLC

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
tables = plc.get_watch_tables()
```

2.2.20. get_technology_objects(folder_path: Optional[str] = None)

Abrufen der Liste der Technologieobjekte

Returns → `List[TechnologyObject]` Liste der Technologieobjekte der PLC

Parameter	Typ	Beschreibung
folder_path	str	Der Ordnerpfad des Elements

```
tos = plc.get_technology_objects()
```

2.2.21. get_software_units()

Abrufen der Liste der Software Units

Returns → `List[SoftwareUnit]` Liste der Software Units

```
software_units = plc.get_software_units()
```

2.2.22. get_safety_administration()

Abrufen der Safety Administration der PLC

Returns → `SafetyAdministration` Safety Administration der PLC

```
sa = plc.get_safety_administration()
```

2.2.23. import_blocks(import_root_directory: str)

Importieren der Programmbausteine aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners
target_folder_path	str	Optional: Der Target Base Path innerhalb der Group Structure der PLC, wo die importierte Folder Hierarchy neu erstellt wird

```
plc.import_blocks(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Program blocks")
```

2.2.24. import_plc_tags(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der Variablentabellen aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners
target_folder_path	str	Optional: Der Zielordner innerhalb der Gruppenstruktur der PLC, in die die importierte Ordnerhierarchie neu erstellt wird

```
plc.import_plc_tags(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC tags")
```

2.2.25. import_data_types(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der PLC-Datentypen aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners
target_folder_path	str	Optional: Der Zielordner innerhalb der Gruppenstruktur der PLC, in die die importierte Ordnerhierarchie neu erstellt wird

```
plc.import_data_types(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC data types")
```

2.2.26. import_technology_objects(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der Technologieobjekte aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners
target_folder_path	str	Optional: Der Zielordner innerhalb der Gruppenstruktur der PLC, in die die importierte Ordnerhierarchie neu erstellt wird

```
plc.import_technology_objects(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Technology objects")
```

2.2.27. import_watch_tables(import_root_directory: str, target_folder_path: Optional[str] = None)

Importieren der Beobachtungstabellen aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners
target_folder_path	str	Optional: Der Zielordner innerhalb der Gruppenstruktur der PLC, in die die importierte Ordnerhierarchie neu erstellt wird

```
plc.import_watch_tables(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Watch and force tables")
```

2.2.28. create_software_unit(name: str)

Erstellen einer Software Unit in der PLC

Parameter	Typ	Beschreibung
name	str	Name der Software Unit

```
plc.create_software_unit(name = "Unit 1")
```

2.2.29. safety_print(print_file: str)

Erstellen eines Sicherheitsausdruck der PLC und Drucken in eine Datei

Wenn die Datei bereits existiert, wird sie überschrieben.

Returns → `bool` Gibt bei Erfolg `true` zurück

Parameter	Typ	Beschreibung
print_file	str	Vollständiger Pfad der Druckdatei

```
plc.safety_print(print_file = "C:\\ws\\safetyprint\\F_PLC_Printout.pdf")
```

2.2.30. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.2.31. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.2.32. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.2.33. get_idenfier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifizier()
```

2.3. Enums

2.3.1. PortalMode

Enum Element	Wert
WithGraphicalUserInterface	0
WithoutGraphicalUserInterface	1
AnyUserInterface	2

2.3.2. UmacUserMode

Enum Element	Wert
Projekt	0
Global	1

2.3.3. ExportFormats

Enum Element	Wert
SimaticML	0
ExternalSource	1
SimaticSD	2

2.3.4. ExportOptions

Enum Element	Wert
WithDefaults	0
Nan	1
WithReadOnly	2

2.3.5. CleanUpMode

Enum Element	Wert
PreserveDefaultVersionOfUnusedTypes	0
DeleteUnusedTypes	1

2.3.6. LibraryExportOptions

Enum Element	Wert
Nan	0
WithLibraryVersionInfoFile	1
OnlyLibraryVersionInfoFile	2

2.3.7. HarmonizeOptions

Enum Element	Wert
HarmonizePathsAndNames	0
HarmonizePaths	1
HarmonizeNames	2

2.3.8. DependenciesMode

Enum Element	Wert
DoNotAutomaticallyCreateOrReleaseDependencies	0
AutomaticallyCreateOrReleaseDependenciesIfRequired	1

2.4. Global - Global

Funktionen zum Starten oder Verbinden mit TIA Portal sind verfügbar, um die TIA Portal Instance zu erhalten.

2.4.1. **open_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)**

Öffnen einer neuen TIA Portal-Instanz

Versions-Angabe im Format major.minor z.B. "18.0"

Returns → **Portal** TIA Portal Instanz

Parameter	Typ	Beschreibung
portal_mode	Enums.PortalMode	Mit oder ohne Benutzeroberfläche
Version	str	Versions-Angabe von TIA Portal im Format major.minor z.B. 18.0, (Standardmäßig aktuellste installierte TIA Portal Version)

Beispiel für die Verwendung

```
portal = siemens_tia_scripting.open_portal(portal_mode =
siemens_tia_scripting.Enums.PortalMode.WithGraphicalUserInterface, version = "18.0")
```

2.4.2. **attach_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)**

Anhängen an eine laufende TIA Portal-Instanz

Versions-Angabe im Format major.minor z.B. "18.0"

Returns → **Portal** TIA Portal Instanz

Parameter	Typ	Beschreibung
portal_mode	Enums.PortalMode	Mit oder ohne Benutzeroberfläche
Version	str	Versions-Angabe von TIA Portal im Format major.minor z.B. 18.0, (Standardmäßig aktuellste installierte TIA Portal Version)

```
portal = siemens_tia_scripting.attach_portal(portal_mode =
siemens_tia_scripting.Enums.PortalMode.WithGraphicalUserInterface, version = "18.0")
```

2.4.3. **open_attach_project(project_file_path: str, portal_mode: Optional[Enums.PortalMode] = None, server_project_view: Optional[bool] = None)**

Anhängen an eine laufende TIA Portal-Instanz mit bereits geöffnetem Projekt

Oder öffnet das Projekt mit einer neuen Instanz der passenden TIA Portal-Version

Returns → **Projekt** TIA Projekt-Instanz

Parameter	Typ	Beschreibung
project_file_path	str	Vollständiger Pfad der Projektdatei
portal_mode	Enums.PortalMode	Mit oder ohne Benutzeroberfläche
server_project_view	str	Standard: <code>false</code> . Wenn der Parameter auf <code>true</code> gesetzt ist, wird die Server Project View geöffnet.

```
project = siemens_tia_scripting.open_attach_project(project_file_path =
"C:\\ws\\testproj\\testproj.ap17", portal_mode =
siemens_tia_scripting.Enums.PortalMode.WithGraphicalUserInterface)
```


2.4.4. **get_installed_bundles()**

Abrufen der Liste der installierten TIA Portal-Bundles

Returns → `List[ProductBundle]` Liste der installierten Bundles

```
product_bundles = siemens_tia_scripting.get_installed_bundles()
```

2.4.5. **get_installed_products()**

Abrufen der Liste der installierten TIA Portal Produkte

Returns → `List[Product]` Liste der installierten Produkte

```
products = siemens_tia_scripting.get_installed_products()
```

2.4.6. **set_umac_credentials(user_name: str, user_password: str, user_type: Enums.UmacUserMode)**

Setzen der UMAC Zugangsdaten, die für geschützte Bibliotheken oder geschützte Projekte verwendet werden sollen

Parameter	Typ	Beschreibung
user_name	str	Benutzername
user_password	str	Passwort des Benutzers
user_type	Enums.UmacUserMode	Projekt- oder Globaler Benutzer

```
siemens_tia_scripting.set_umac_credentials(user_name = "admin", user_password = "Password123",  
user_type = siemens_tia_scripting.Enums.UmacUserMode.Project)
```

2.4.7. **encrypt_umac_config(umac_file_path: str, secret: str, secret_env_name: str)**

Verschlüsseln der UMAC und UMC Konfiguration aus dem TIA Portal Projekt mit dem bereitgestellten Secret.

Parameter	Typ	Beschreibung
umac_file_path	str	Vollständiger Pfad der UMAC Konfigurationsdatei
secret	str	Secret für die Verschlüsselung
secret_env_name	str	Name der Umgebungsvariablen, in der das Secret gespeichert ist

```
siemens_tia_scripting.encrypt_umac_config(umac_file_path =  
"C:\\ws\\exportfolder\\exportUMAC.json", secret = "mySecret")
```

2.4.8. **set_logging(path: str, console: bool)**

Festlegen des Logging-Ausgabepfad oder Konsolenausgabe (stdout)

Parameter	Typ	Beschreibung
path	str	Vollständiger Pfad der Protokolldatei
console	bool	Optional: True für die Aktivierung der Konsolenausgabe, False zum Deaktivieren der Standardausgabe im Konsolenfenster

```
siemens_tia_scripting.set_logging(path = "c:\\ws\\tiascripting.log", console = False)
```

2.5. Global - Portal

Das Portal-Objekt stellt eine TIA Portal-Instanz dar und bietet Funktionen zur Verwaltung von Projekten und Bibliotheken in TIA Portal.

2.5.1. get_process_id()

Abrufen der TIA Portal Prozess-ID

Returns → `int` TIA Portal Prozess-ID

```
id = portal.get_process_id()
```

2.5.2. open_project(project_file_path: str, server_project_view: Optional[bool] = None)

Öffnen eines TIA Portal Projekts oder einer lokale Session - optional in Serverprojektansicht

Returns → `Projekt` TIA Portal Projekt

Parameter	Typ	Beschreibung
project_file_path	str	Vollständiger Pfad der Projektdatei
server_project_view	str	Standard: false. Wenn Parameter auf true gesetzt, öffne es in der Serverprojektansicht

```
project = portal.open_project(project_file_path = "C:\\ws\\testproj\\testproj.ap17")
```

2.5.3. open_project_with_copy(project_file_path: str, target_directory_path: str, delete_existing_project: bool)

Öffnen eines TIA Portal Projekts oder einer lokale Session in anderem Ordner

Returns → `Projekt` TIA Portal Projekt

Parameter	Typ	Beschreibung
project_file_path	str	Vollständiger Pfad der Projektdatei
target_directory_path	str	Temporärer Pfad, in den das Projekt vor dem Öffnen kopiert werden soll
delete_existing_project	bool	Legt fest, ob das temporäre Projekt gelöscht werden soll

```
project = portal.open_project_with_copy(project_file_path = "C:\\ws\\testproj\\testproj.ap17",
target_directory_path = "C:\\ws\\temp" , delete_existing_project = True)
```

2.5.4. retrieve_archive(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)

Entpacken eines TIA Portal-Projektarchivs

Returns → `Projekt` TIA Portal Projekt

Parameter	Typ	Beschreibung
target_directory_path	str	Temporärer Pfad, in den das Projekt vor dem Öffnen kopiert werden soll
archive_file_path	str	Vollständiger Pfad der archivierten Projektdatei
delete_existing_project	bool	Legt fest, ob das temporäre Projekt gelöscht werden soll

```
project = portal.retrieve_archive(archive_file_path = "C:\\ws\\testproj\\testproj.zap17",
target_directory_path = "C:\\ws\\temp" , delete_existing_project = True)
```

2.5.5. **create_project(target_directory_path: str, project_name: str, delete_existing_project: bool)**

Erstellen eines neuen TIA Portal-Projekts

Returns → **Projekt** TIA Portal Projekt

Parameter	Typ	Beschreibung
target_directory_path	str	Temporärer Pfad, in den das Projekt vor dem Öffnen kopiert werden soll
project_name	str	Name des neuen Projekts
delete_existing_project	bool	Legt fest, ob das temporäre Projekt gelöscht werden soll

```
project = portal.create_project(target_directory_path = "C:\\ws\\temp" , project_name =
"MyNewProject" delete_existing_project = True)
```

2.5.6. **get_project()**

Abrufen des geöffneten TIA Portal-Projekt der aktuellen TIA Portal-Instanz

Returns → **Projekt** TIA Portal Projekt

```
project = portal.get_project()
```

2.5.7. **get_global_library_infos()**

Abrufen einer Liste der Globalen Bibliotheks Informationen der aktuellen TIA Portal Instanz

Returns → **List[GlobalLibraryInfo]** Liste der Globalen Bibliotheks Informationen

```
global_lib_infos = portal.get_global_library_infos()
for info in global_lib_infos:
...
```

2.5.8. **get_global_library(library_name: str)**

Abrufen einer globalen Bibliothek

Returns → **GlobalLibrary** Globale Bibliothek

Parameter	Typ	Beschreibung
library_name	str	Name der globalen Bibliothek

```
global_lib = portal.get_global_library(library_name = "GlobalLib1")
```

2.5.9. **open_global_library(library_path: str)**

Öffnen einer globalen Bibliothek

Returns → **GlobalLibrary** Globale Bibliothek

Parameter	Typ	Beschreibung
library_path	str	Vollständiger Pfad der globalen Bibliothek

```
global_lib = portal.open_global_library(library_path = "C:\\ws\\testlib\\testlib.al17")
```

2.5.10. **open_global_library_with_copy(target_directory_path: str, library_path: str, delete_existing_project: bool)**

Öffnen einer globalen Benutzerbibliothek

Returns → `GlobalLibrary` Globale Bibliothek

Parameter	Typ	Beschreibung
target_directory_path	str	Temporärer Pfad, in den die Bibliothek vor dem Öffnen kopiert werden soll
library_path	str	Vollständiger Pfad der globalen Bibliothek
delete_existing_project	bool	Legt fest, ob das temporäre Projekt gelöscht werden soll

```
global_lib = portal.open_global_library_with_copy(target_directory_path = "C:\\ws\\temp",
library_path = "C:\\ws\\testlib\\testlib.al17", delete_existing_project = True)
```

2.5.11. **create_global_library(target_directory_path: str, library_name: str, delete_existing_project: bool)**

Erstellen einer neuen TIA Portal Bibliothek

Returns → `GlobalLibrary` Globale Bibliothek

Parameter	Typ	Beschreibung
target_directory_path	str	Temporärer Pfad, wohin die Bibliothek vor dem Öffnen kopiert werden soll
library_name	str	Name der neuen Globalen Bibliothek
delete_existing_project	bool	Definiert, ob die temporäre Bibliothek gelöscht werden soll

```
global_lib = portal.create_global_library(target_directory_path = "C:\\ws\\temp" , library_name
= "MyLib1" delete_existing_project = True)
```

2.5.12. **retrieve_archive_library(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)**

Entpacken eines TIA Portal Bibliotheksarchivs

Returns → `GlobalLibrary` Globale Bibliothek

Parameter	Typ	Beschreibung
target_directory_path	str	Temporärer Pfad, in den die Bibliothek vor dem Öffnen kopiert werden soll
archive_file_path	str	Vollständiger Pfad der archivierten Bibliotheksdatei
delete_existing_project	bool	Legt fest, ob das temporäre Projekt gelöscht werden soll

```
global_lib = portal.retrieve_archive_library(target_directory_path = "C:\\ws\\temp",
archive_file_path = "C:\\ws\\testproj\\testlib.zal17", delete_existing_project = True)
```

2.5.13. **close_portal()**

Schließen der TIA Portal-Instanz (erfordert, dass alle Projects gespeichert oder Änderungen explizit verworfen werden)

```
portal.close_portal()
```

2.5.14. show_online_finger_prints(mode: str, pci_interface: str, ip_address: str)

Anzeigen der Fingerprints der verbundenen PLC

Parameter	Typ	Beschreibung
mode	str	Konfigurationsmodus
pci_interface	str	PC Schnittstellen Name
ip_address	str	IP-Adresse

```
portal.show_online_finger_prints(mode = "PN/IE", pci_interface = "PLCSIM", ip_address = "192.168.0.10")
```

2.5.15. get_project_servers(server: Optional[str] = None)

Abrufen der Liste von TIA Projekt-Server, die über einen Host-Filter gefunden wurden

Returns → List[ProjectServer] Liste der TIA Project-Server

Parameter	Typ	Beschreibung
server	str	Server Name oder URL

```
portal.get_project_servers(server = "CompanyServer")
```

2.5.16. detach()

Trennen von der TIA Portal-Instanz

```
portal.detach()
```

2.5.17. add_project_server(url: str, name: str)

Hinzufügen eines neuen TIA Project-Server

Returns → ProjectServer TIA Project-Server

Parameter	Typ	Beschreibung
url	str	URL des TIA Project-Servers
name	str	Name des neuen Servers

```
portal.add_project_server(url = "https://project.server.net:8735/", name = "MyNewServer")
```

2.6. Global - Product

Das Product-Objekt repräsentiert ein Objekt der installierten Siemens Produktsoftware.

2.6.1. **get_name()**

Abrufen des Namens des TIA Portal-Produkts

Returns → **str** Name des TIA Portal-Produkts

```
product_name = product.get_name()
```

2.6.2. **get_release()**

Abrufen des Releases des TIA Portal-Produkts

Returns → **str** Release-Tag des TIA Portal-Produkts

```
product_release = product.get_release()
```

2.6.3. **get_version()**

Abrufen der Version des TIA Portal-Produkts

Returns → **str** Version des TIA Portal-Produkts

```
product_version = product.get_version()
```

2.7. Global - ProductBundle

Das ProductBundle-Objekt stellt ein Objekt der installierten Siemens-Bundle-Software dar.

2.7.1. `get_title()`

Abrufen des Titels des TIA Portal-Bundles

Returns → `str` Titel des TIA Portal-Bundles

```
bundle_title = bundle.get_title()
```

2.7.2. `get_release()`

Abrufen des Releases des TIA Portal-Bundles

Returns → `str` Release des TIA Portal-Bundles

```
bundle_release = bundle.get_release()
```

2.7.3. `get_products()`

Abrufen der Liste der Produkte des TIA Portal-Bundles

Returns → `List[Product]` Liste der TIA Portal-Bundles

```
bundle_products = bundle.get_products()
```

2.8. HMI Data - HmiAlarm

Die Klasse repräsentiert einen HMI Alarm in TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über HMI Alarmer und zum Abrufen von Eigenschaften.

2.8.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.8.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.8.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.8.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.8.5. get_identifier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```


2.9. HMI Data - HmiAlarmClass

Die Klasse repräsentiert eine HMI Alarm Class im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über die HMI Alarm Class und zum Abrufen von Properties.

2.9.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.9.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.9.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.9.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.9.5. get_identifier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.10. HMI Data - HmiConnection

Die Klasse repräsentiert eine HMI Connection im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über die HMI Connection und zum Abrufen von Properties.

2.10.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.10.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.10.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.10.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.10.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.10.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.11. HMI Data - HmiCycle

Die Klasse repräsentiert einen HMI Cycle im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über den HMI Cycle und zum Abrufen von Properties.

2.11.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.11.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.11.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.11.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.11.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.11.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.12. HMI Data - HmiGraphicList

Die Klasse repräsentiert eine HMI GraphicList im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über die HMI GraphicList und zum Abrufen von Properties.

2.12.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.12.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.12.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.12.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.12.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.12.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.13. HMI Data - HmiScreen

Die Klasse repräsentiert einen HMI Screen im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über den HMI Screen und zum Abrufen von Properties.

2.13.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.13.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.13.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.13.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.13.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.13.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.14. HMI Data - HmiScript

Die Klasse repräsentiert ein HMI Script im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über das HMI Script und zum Abrufen von Properties.

2.14.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.14.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.14.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.14.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.14.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.14.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.15. HMI Data - HmiTag

Die Klasse repräsentiert einen HMI Tag im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über den HMI Tag und zum Abrufen von Properties.

2.15.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.15.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.15.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.15.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.15.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.15.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.16. HMI Data - HmiTagTable

Die Klasse repräsentiert eine HMI TagTable im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über die HMI TagTable und zum Abrufen von Properties.

2.16.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.16.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.16.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.16.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.16.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.16.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.17. HMI Data - HmiTextList

Die Klasse repräsentiert eine HMI TextList im TIA Portal. Sie bietet Methoden zum Abrufen von Informationen über die HMI TextList und zum Abrufen von Properties.

2.17.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = tiap_object.get_name()
```

2.17.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.17.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.17.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.17.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.17.6. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.18. Library - GlobalLibrary

Die Klasse bietet Methoden zur Interaktion mit globalen TIA Portal-Bibliotheken. Sie ermöglicht es, Informationen über die Bibliothek abzurufen, sie zu speichern, den Autor abzurufen, zu prüfen, ob sie geändert und schreibgeschützt ist, und verschiedene Operationen mit der Bibliothek durchzuführen.

2.18.1. get_name()

Abrufen des Namens der globalen Bibliothek

Returns → `str` Name der globalen Bibliothek

```
name = global_lib.get_name()
```

2.18.2. save()

Speichern der globalen Bibliothek

```
global_lib.save()
```

2.18.3. get_author()

Abrufen des Autors der globalen Bibliothek

Returns → `str` Name des Autors

```
author = global_lib.get_author()
```

2.18.4. get_path()

Abrufen des Pfads der globalen Bibliothek

Returns → `str` Vollständiger Pfad der Bibliothek

```
path = global_lib.get_path()
```

2.18.5. is_modified()

Prüfen auf Änderungen der globalen Bibliothek

Returns → `bool` Status, ob die Bibliothek geändert wurde

```
state = global_lib.is_modified()
```

2.18.6. is_read_only()

Prüfen auf Schreibschutz der globalen Bibliothek

Returns → `bool` Zustand, wenn die globale Bibliothek schreibgeschützt ist

```
state = global_lib.is_read_only()
```

2.18.7. update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)

Aktualisieren der Zielbibliothek mit Typ(en) aus der Quelle; wenn keine Zielbibliothek ausgewählt ist, wird die Projektbibliothek aktualisiert

Parameter	Typ	Beschreibung
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3:

Parameter	Typ	Beschreibung
		'SetOnlyHigherUpdatedVersionAsDefault']
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancellfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Typen auszuwählen. Wenn leer, werden alle Typen der Projektbibliothek ausgewählt
library_name	str	Optional: Diese Option sollte verwendet werden, wenn eine spezifische Globale Bibliothek Library aktualisiert werden muss. Wenn kein Name angegeben ist, wird die Projektbibliothek des aktuellen Projekts ausgewählt

```
global_lib.update_library(update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"], library_name = "MyGlobalLibrary")
```

2.18.8. update_project(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)

Aktualisieren des aktuellen Projekts mit Typ(en) aus der Bibliothek

Parameter	Typ	Beschreibung
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3: 'SetOnlyHigherUpdatedVersionAsDefault']
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancellfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Source Type oder Types auszuwählen. Wenn leer, werden alle Types in der Project Library ausgewählt

```
global_lib.update_project(update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"])
```

2.18.9. harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)

Harmonisieren der Typ-Instanzen im Projekt mithilfe der Globalen Bibliothek. Wenn Typ-GUIDs angegeben sind, werden nur die ausgewählten Typen für das Update verwendet.

Parameter	Typ	Beschreibung
harmonize_options	integer (enum)	[0: 'HarmonizePathsAndNames', 1: 'HarmonizePaths', 2: 'HarmonizeNames']

Parameter	Typ	Beschreibung
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Typen auszuwählen. Wenn leer, werden alle Typen in der Projektbibliothek genutzt

```
global_lib.harmonize_project(harmonize_options: 0, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```

2.18.10. archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)

Archivieren der TIA Portal-Bibliothek

Parameter	Typ	Beschreibung
target_directory_path	str	Pfad des Ordners, in dem das Archiv abgelegt werden soll
archive_name	str	Name der Archivdatei
delete_existing_archive	bool	Legt fest, ob die Ausgabedatei zuerst gelöscht werden soll

```
global_lib.archive(target_directory_path = "C:\\ws\\newfolder", archive_name = "testglobalib", delete_existing_archive = True)
```

2.18.11. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → **str** Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.18.12. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → **List[str]** Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.18.13. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → **int** Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts

Parameter	Typ	Beschreibung
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.18.14. **clean_up(folder_path: Optional[str] = None)**

Aufräumen der Bibliothek, ohne alle Typen zu löschen

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek, in dem die importierte Ordnerhierarchie neu erstellt wird

```
global_lib.clean_up()
```

2.18.15. **get_types()**

Abreufen der Bibliotheks-Typen des Ordners

Returns → `List[LibraryType]` Liste der Typen

```
types = global_lib.get_types()
```

2.18.16. **find_library_type(library_type_name: str)**

Suchen des Bibliotheks-Typ anhand des Namen

Returns → `LibraryType` Bibliotheks-Typ

Parameter	Typ	Beschreibung
library_type_name	str	Name des Bibliotheks-Typ

```
type = global_lib.find_library_type(library_type_name = "MyType")
```

2.18.17. **close_global_library()**

Schließen der Globalen Bibliothek

```
global_lib.close_global_library()
```

2.18.18. **create_folder(folder_path: str)**

Erstellen eines neuen Ordners im angegebenen Pfad in der Bibliothek

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek

```
global_lib.create_folder(folder_path = "myFolder/subFolderExample")
```

2.18.19. **delete_folder(folder_path: str)**

Löschen des angegebenen Ordners in der Bibliothek

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek

```
global_lib.delete_folder(folder_path = "myFolder/subFolderExample")
```

2.19. Library - GlobalLibraryInfo

Die Klasse stellt Informationen über eine globale Bibliothek im Kontext von TIA Portal dar.

2.19.1. get_name()

Abrufen des Namens der globalen Bibliotheksinformation

Returns → `str` Name der globalen Bibliotheksinformation

```
name = global_lib_info.get_name()
```

2.19.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.19.3. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.19.4. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.20. Library - LibraryType

Die Klasse repräsentiert einen Bibliothekstyp im Kontext von TIA Portal.

2.20.1. get_name()

Abrufen des Namens des Bibliothekstyps

Returns → `str` Name des Typs

```
name = lib_type.get_name()
```

2.20.2. get_author()

Abrufen des Autors des Bibliothekstyps

Returns → `str` Autor des Typs

```
author = lib_type.get_author()
```

2.20.3. get_guid()

Abrufen der GUID des Bibliothekstyps

Returns → `str` GUID des Typs

```
guid = lib_type.get_guid()
```

2.20.4. get_versions()

Abrufen der Versionen des Bibliothekstyps

Returns → `List[LibraryTypeVersion]` Versionen des Typs

```
versions = lib_type.get_versions()
```

2.20.5. find_version(version: str)

Suchen nach der Version des Bibliothekstyps

Returns → `str` Version des Typs

```
typeversion = lib_type.find_version(version = "1.0.0")
```

2.20.6. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.20.7. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften


```
properties = tiap_object.get_properties()
```

2.20.8. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.20.9. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → `str` Vollständiger Ordnerpfad

```
group_path = lib_type.get_path_full()
```

2.20.10. get_path()

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → `str` Ordnerpfad

```
group_path = lib_type.get_path_full()
```

2.20.11. get_comment()

Abrufen des Kommentar-Texts. Die Sprache wird durch die aktuelle Editiersprache festgelegt.

Returns → `str` Kommentar-Text

```
comment = lib_type.get_comment()
```

2.20.12. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.21. Library - LibraryTypeFolder

Die Klasse stellt einen Ordner dar, der Bibliothekstypen und Unterordner im Kontext von TIA Portal enthält.

2.21.1. get_name()

Abrufen des Namen des Ordners

Returns → `str` Name des Ordners

```
name = lib_folder.get_name()
```

2.21.2. get_folders()

Abrufen der Bibliothekstyp-Unterordner

Returns → `List[LibraryTypeFolder]` Liste von Unterordnern

```
folders = lib_folder.get_folders()
```

2.21.3. get_types()

Abrufen der Bibliothekstypen des Ordners

Returns → `List[LibraryType]` Liste der Typen

```
types = lib_folder.get_types()
```

2.21.4. find_library_type(library_type_name: str)

Suchen der Bibliothekstypen anhand des Namens

Returns → `LibraryType` Bibliothekstyp

Parameter	Typ	Beschreibung
library_type_name	str	Name des Bibliothekstyps

```
type = lib_folder.find_library_type(library_type_name = "MyType")
```

2.21.5. find_folder(folder_name: str)

Suchen der Bibliothekstyp-Ordner in Unterordnern

Returns → `LibraryTypeFolder` Bibliothekstyp-Ordner

Parameter	Typ	Beschreibung
folder_name	str	Name des Unterordners

```
folder = lib_folder.find_folder(folder_name = "MyFolder")
```

2.22. Library - LibraryTypeVersion

Die Klasse stellt eine Version eines Bibliothekstyps im Kontext von TIA Portal dar.

2.22.1. get_author()

Abrufen des Autors der Bibliothekstypversion

Returns → `str` Autor

```
author = projectlib.get_author()
```

2.22.2. get_guid()

Abrufen der GUID der Bibliothekstypversion

Returns → `str` GUID

```
guid = lib_type_version.get_guid()
```

2.22.3. get_version_number()

Abrufen der Versionsnummer der Bibliothekstypversion

Returns → `str` Versionsnummer

```
version_number = lib_type_version.get_version_number()
```

2.22.4. get_modified_date()

Abrufen des Änderungsdatums der Bibliothekstypversion

Returns → `str` Änderungsdatum

```
date = lib_type_version.get_modified_date()
```

2.22.5. get_state()

Abrufen des Status der Bibliothekstypversion

Returns → `str` Status der Version des Bibliothekstyps

```
state = lib_type_version.get_state()
```

2.22.6. get_type_object()

Abrufen des Typen der Bibliothekstypversion

Returns → `LibraryType` Bibliothekstyp-Objekt

```
lib_type = lib_type_version.get_type_object()
```

2.22.7. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.22.8. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.22.9. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.22.10. find_instances(device_name: Optional[str] = None)

Abrufen der Liste von Instanzen der Bibliotheks-Typ-Version

Returns → `List[object]` Liste der Instanzen

Parameter	Typ	Beschreibung
device_name	str	Optional: Name des Geräts, in dem die Instanzen gesucht werden sollen (z.B. PLC oder HMI)

```
blocks = lib_type_version.find_instances()
```

2.22.11. instantiate(device_name: str, software_unit_name: Optional[str] = None, folder_path: Optional[str] = None)

Erstellen einer Instanz in der PLC oder HMI mit der aktuellen Typ-Version

Parameter	Typ	Beschreibung
device_name	str	Name der PLC Software oder HMI Software, z.B. HMI_RT_1
software_unit_name	str	Optional: Name der PLC Software Unit
folder_path	str	Optional: Ordnerstruktur, z.B. group1/group2

```
lib_type_version.instantiate(device_name = "PLC_1", software_unit_name: "Unit1", folder_path: "group1/group2")
```

2.22.12. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = lib_type_version.get_supported_export_format()
```

2.22.13. **export(target_directory_path: str, export_format: str, library_export_options: Enums.LibraryExportOptions, keep_folder_structure: Optional[bool] = None)**

Exportieren des Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
<code>target_directory_path</code>	<code>str</code>	Ordnerpfad für den Export
<code>export_options</code>	<code>Enums.ExportOptions</code>	Optional: Exportoptionen, Standard ist <code>ExportOptions.None</code>
<code>export_format</code>	<code>Enums.ExportFormats</code>	Optional: Exportformat, Standard ist <code>SimaticML</code>
<code>keep_folder_structure</code>	<code>bool</code>	<code>True</code> : Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
lib_type_version.export(library_export_options =
    ts.Enums.LibraryExportOptions.WithLibraryVersionInfoFile, export_format = "SimaticML",
    keep_folder_structure = true)
```

2.22.14. **edit(device_name: Optional[str] = None)**

Setzen des Bibliotheks-Typ in den Bearbeitungsmodus für ein spezifisches Gerät

Returns → `LibraryTypeVersion` Das bearbeitete Bibliotheks-Typ-Version Objekt.

Parameter	Typ	Beschreibung
<code>device_name</code>	<code>str</code>	Optional: Name des Geräts, in dem die Instanz gesucht werden soll (z.B. PLC oder HMI)

```
edited_version = lib_type_version.edit(device_name="PLC_1")
```

2.22.15. **release(dependencies_mode: Enums.DependenciesMode, version: str, author: str, comment: str)**

Freigeben (Release) der Bibliotheks-Typ-Version

Parameter	Typ	Beschreibung
<code>dependencies_mode</code>	<code>Enums.DependenciesMode</code>	Mode für Dependencies
<code>version</code>	<code>str</code>	Versions-Zeichenfolge
<code>author</code>	<code>str</code>	Name des Autors
<code>comment</code>	<code>str</code>	Freigabekommentar

```
lib_type_version.release(dependencies_mode=ts.Enums.DependenciesMode.AutomaticallyCreateOrReleaseDependenciesIfRequired, version="1.0.0", author="John", comment="Initial release")
```

2.22.16. **discard()**

Verwerfen (Discard) der Änderungen an der Bibliotheks-Typ-Version

```
lib_type_version.discard()
```

2.22.17. **set_as_default()**

Setzen der Bibliotheks-Typ-Version als Standard

```
lib_type_version.set_as_default()
```

2.22.18. **get_comment()**

Abrufen des Kommentars für die Bibliotheks-Typ-Version

Returns → **str** Kommentar-Text

```
comment = lib_type_version.get_comment()
```

2.22.19. **get_identifizier()**

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.23. Library - ProjectLibrary

Die Klasse stellt eine Projektbibliothek im Kontext von TIA Portal dar.

2.23.1. get_type_folder()

Abrufen des Ordners, der Bibliothekstypen und Bibliothekstyp-Ordner enthält

Returns → `LibraryTypeFolder` Bibliothekstyp-Ordner

```
typefolder = projectlib.get_type_folder()
```

2.23.2. find_library_type(library_type_name: str)

Suchen des Bibliotheks-Typ anhand des Namen

Returns → `LibraryType` Bibliotheks-Typ

Parameter	Typ	Beschreibung
library_type_name	str	Name des Bibliotheks-Typ

```
type = global_lib.find_library_type(library_type_name = "MyType")
```

2.23.3. clean_up(folder_path: Optional[str] = None)

Aufräumen der Bibliothek, ohne alle Typen zu löschen

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek, in dem die importierte Ordnerhierarchie neu erstellt wird

```
global_lib.clean_up()
```

2.23.4. import_library_types(import_root_directory: str, device_name: Optional[str] = None, software_unit_name: Optional[str] = None, plc_folder_path: Optional[str] = None, library_folder_path: Optional[str] = None)

Importieren von Bibliotheks-Typen aus einem Verzeichnis in die Projektbibliothek

Parameter	Typ	Beschreibung
import_root_directory	str	Der Verzeichnispfad, von dem alle Dateien und Ordner importiert werden
device_name	str	Optional: Name der PLC Software
software_unit_name	str	Optional: Name der PLC Software Unit der PLC
plc_folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Ordnerstruktur der PLC, in dem die importierte Ordnerhierarchie neu erstellt wird
library_folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Projektbibliothek, in dem die importierte Ordnerhierarchie neu erstellt wird

```
projectlib.import_library_types(import_root_directory= "C:\\ws\\importLibraryTypes",
device_name: "PLC_1", software_unit_name: "Unit1", plc_folder_path: "group1/group2",
library_folder_path: "ImportLibFolder/Types")
```

2.23.5. **update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)**

Aktualisieren der Zielbibliothek mit Typ(en) aus der Quelle

Parameter	Typ	Beschreibung
library_name	str	Der Name der Bibliothek, die aktualisiert werden soll
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3: 'SetOnlyHigherUpdatedVersionAsDefault']
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancelIfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Typen auszuwählen. Wenn leer, werden alle Typen der Projektbibliothek ausgewählt

```
project_lib.update_library(library_name = "MyGlobalLibrary", update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"])
```

2.23.6. **update_project(delete_mode: int, type_guids: Optional[List[str]] = None)**

Aktualisieren des aktuellen Projekts mit Typ(en) aus der Bibliothek. Wenn Typ-GUIDs angegeben sind, werden nur die ausgewählten Typen für das Update verwendet.

Parameter	Typ	Beschreibung
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Source Type oder Types auszuwählen. Wenn leer, werden alle Types in der Project Library ausgewählt

```
project_lib.update_library(delete_mode: 1, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```

2.23.7. **harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)**

Harmonisieren der Typ-Instanzen im Projekt mithilfe der Projektbibliothek. Wenn Typ-GUIDs angegeben sind, werden nur die ausgewählten Typen für das Update verwendet.

Parameter	Typ	Beschreibung
harmonize_options	integer (enum)	[0: 'HarmonizePathsAndNames', 1: 'HarmonizePaths', 2: 'HarmonizeNames']
type_guids	List[str]	Optional: Diese Option sollte verwendet werden, um spezifische Typen auszuwählen. Wenn leer, werden alle Typen in der Projektbibliothek genutzt


```
project_lib.harmonize_project(harmonize_options: 0, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```

2.23.8. create_folder(folder_path: str)

Erstellen eines neuen Ordners im angegebenen Pfad in der Bibliothek

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek

```
projectlib.create_folder(folder_path = "myFolder/subFolderExample")
```

2.23.9. delete_folder(folder_path: str)

Löschen des angegebenen Ordners in der Bibliothek

Parameter	Typ	Beschreibung
folder_path	str	Optional: Der Basis-Zielpfad innerhalb der Bibliothek

```
projectlib.delete_folder(folder_path = "myFolder/subFolderExample")
```

2.24. PLC Data - ExternalSource

Die Klasse stellt eine externe Quelle in TIA Portal dar. Sie bietet Methoden zum Abrufen von Informationen über externe Quellen, zum Abrufen von Eigenschaften und zum Erzeugen von Bausteinen.

2.24.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.24.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.24.3. block_gen()

Generieren der Bausteine der externen Quelle

```
ext_source.block_gen()
```

2.24.4. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.24.5. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.24.6. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts

Parameter	Typ	Beschreibung
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.24.7. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = lib_type_version.get_supported_export_format()
```

2.24.8. **get_identifier()**

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.25. PLC Data - ForceTable

Die Klasse stellt eine Forcetabelle im TIA Portal dar. Sie bietet Methoden zum Abrufen von Informationen über die Forcetabelle, zum Abrufen von Eigenschaften und zum Exportieren der Tabelle.

2.25.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.25.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.25.3. export(target_directory_path: str, export_options: Enums.ExportOptions, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf true gesetzt ist, werden alle Gruppennamen hierarchisch in Ordnern abgelegt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Exportoptionen
export_format	Enums.ExportFormats	Exportformat, standardmäßig SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch in Ordnern abgelegt

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.25.4. is_consistent()

Prüfen der Konsistenz der Forcetabelle

Returns → `bool` Wahr, wenn konsistent

```
value = force_table.is_consistent()
```

2.25.5. show_in_editor()

Öffnen des PLC-Objekts im Editor

```
plc_object.show_in_editor()
```

2.25.6. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.25.7. **set_property(name:str, value: str)**

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.25.8. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = lib_type_version.get_supported_export_format()
```

2.25.9. **get_path_full()**

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → `str` Vollständiger Ordnerpfad

```
group_path = lib_type.get_path_full()
```

2.25.10. **get_path()**

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → `str` Ordnerpfad

```
group_path = lib_type.get_path_full()
```

2.25.11. **get_fingerprints()**

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.25.12. **get_identifier()**

Abrufen des Identifiers des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifizier()
```

2.26. PLC Data - NamedValueType

Die Klasse repräsentiert einen Named value-Datentyp (NVT) in TIA Portal. Sie bietet Methoden zur Verwaltung und Steuerung von Named value-Datentypen, zum Exportieren des Datentyps, zum Abrufen von Datentypinformationen und zur Überprüfung von Querverweisen.

2.26.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = nvt.get_name()
```

2.26.2. get_namespace()

Abrufen des Namens des Namespace des Named value-Datentyp

Returns → `str` Name des Namespaces

```
name = nvt.get_namespace()
```

2.26.3. export(target_directory_path: str, keep_folder_structure: Optional[bool] = None)

Exportieren des benannten Werttyps

Wenn `keep_folder_structure` auf `true` gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
<code>target_directory_path</code>	<code>str</code>	Ordnerpfad für den Export
<code>keep_folder_structure</code>	<code>bool</code>	True: Alle Gruppennamen werden hierarchisch in Ordnern exportiert

```
nvt.export(target_directory_path = "C:\\ws\\export", keep_folder_structure = True)
```

2.27. PLC Data - PlcTag

Die Klasse bietet Methoden zur Interaktion mit TIA Portal PLC-Variablen. Sie ermöglicht es, den Namen und die Eigenschaften abzurufen und Exportvorgänge für PLC-Variablen durchzuführen.

2.27.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.27.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.27.3. export(target_directory_path: str, export_options: Enums.ExportOptions, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Exportoptionen
export_format	Enums.ExportFormats	Exportformat, standardmäßig SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.27.4. delete()

Löschen des Objekts

```
plc_object.delete()
```

2.27.5. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.27.6. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.27.7. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = lib_type_version.get_supported_export_format()
```

2.27.8. **get_identifier()**

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.28. PLC Data - PlcTagTable

Die stellt eine Tabelle von PLC-Variablen in TIA Portal dar. Sie bietet Methoden zur Verwaltung und Steuerung von PLC-Variablen, zum Exportieren der Tabelle, zum Abrufen von Variablen und PLC-Anwenderkonstanten und zur Überprüfung von Querverweisen.

2.28.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.28.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.28.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf true gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.28.4. get_plc_tags()

Abrufen der Liste der PLC-Variablen

Returns → `List[PlcTag]` Liste der PLC-Variablen

```
plc_tags = table.get_plc_tags()
```

2.28.5. get_user_constants()

Abrufen der Liste der PLC-Anwenderkonstanten

Returns → `List[UserConstant]` Liste der PLC-Anwenderkonstanten

```
constants = table.get_user_constants()
```

2.28.6. export_cross_references(target_directory_path: str, filter: int)

Exportieren der Querverweisen aus der PLC-Variablentabelle filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
table.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences", filter = 2)
```

2.28.7. show_in_editor()

Öffnen des PLC-Objekts im Editor

```
plc_object.show_in_editor()
```

2.28.8. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.28.9. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → List[str] Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.28.10. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → int Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.28.11. get_supported_export_format()

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.28.12. `get_path_full()`

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → `str` Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.28.13. `get_path()`

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → `str` Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.28.14. `get_fingerprints()`

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.28.15. `get_identifizier()`

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.29. PLC Data - ProgramBlock

Dies stellt einen Programmbaustein in TIA Portal dar. Es bietet verschiedene Methoden zur Verwaltung und Steuerung von Programmbausteinen.

2.29.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.29.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.29.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.29.4. compile()

Übersetzen des Programmbausteins

```
programblock.compile()
```

2.29.5. is_consistent()

Prüfen der Konsistenz des Programmbausteins

Returns → `bool` True, wenn konsistent

```
value = programblock.is_consistent()
```

2.29.6. export_cross_references(target_directory_path: str, filter: int)

Exportieren der Querverweise des Programmbausteins

filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
programblock.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.29.7. show_in_editor()

Öffnen des PLC-Objekts im Editor

```
plc_object.show_in_editor()
```

2.29.8. get_type_version_guid()

Abrufen der GUID der Typversion

Returns → str GUID

```
guid = plc_object.get_type_version_guid()
```

2.29.9. get_type_guid()

Abrufen der GUID des Typs

Returns → str GUID

```
guid = plc_object.get_type_guid()
```

2.29.10. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.29.11. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → List[str] Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.29.12. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → str Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.29.13. **get_path()**

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → **str** Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.29.14. **set_property(name:str, value: str)**

Setzen der Eigenschaft des Objekts

Returns → **int** Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.29.15. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → **List[str]** Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.29.16. **get_fingerprints()**

Abrufen der Fingerprint-Daten des Objekts

Returns → **List[str]** Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.29.17. **is_library_type()**

Überprüfen ob das Objekt ein Bibliotheks-Typ ist

Returns → **bool** True, wenn es ein Bibliotheks-Typ ist

```
value = plcdata.is_library_type()
```

2.29.18. **set_know_how_protection(password: str)**

Setzen des Know-How-Schutzes für den Programmbaustein mit dem angegebenen Passwort

Parameter	Typ	Beschreibung
password	str	Passwort für Know-How-Schutz

```
programblock.set_know_how_protection(password = "Password!123")
```

2.29.19. remove_know_how_protection(password: str)

Entfernen des Know-How-Schutzes für den Programmbaustein mit dem angegebenen Passwort

Parameter	Typ	Beschreibung
password	str	Passwort für Know-How-Schutz

```
programblock.remove_know_how_protection(password = "Password!123")
```

2.29.20. get_identifizier()

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```


2.30. PLC Data - SafetyAdministration

Die Klasse bietet Methoden zur Interaktion mit der Safety Administration von TIA Portal. Sie ermöglicht es, den Anmeldestatus und den Passwortstatus zu überprüfen sowie Import- und Exportvorgänge durchzuführen.

2.30.1. is_logged_on()

Überprüfen einer aktiven Anmeldung beim Safety Administration des Offline-Programms

Returns → `bool` Status der Anmeldung

```
status = safety_admin.is_logged_on()
```

2.30.2. is_password_set()

Überprüfen ob ein Passwort in der Safety Administration für das Offline-Programm festgelegt wurde

Returns → `bool` Status wenn Passwort gesetzt ist

```
status = safety_admin.is_password_set()
```

2.30.3. get_offline_serial_number()

Abrufen der Offline-Seriennummer

Returns → `str` Offline-Seriennummer

```
value = safety_admin.get_offline_serial_number()
```

2.30.4. export_config(target_directory_path: str)

Exportieren der Konfiguration der Safety Administration

Der Ordner "SafetyAdministration" wird automatisch im `target_directory_path` angelegt, falls er noch nicht existiert.

Parameter	Typ	Beschreibung
<code>target_directory_path</code>	<code>str</code>	Ordnerpfad für den Export

```
safety_admin.export_config(target_directory_path = "C:\\ws\\export")
```

2.30.5. import_config(import_root_directory: str)

Importieren der Konfiguration der Safety Administration

Parameter	Typ	Beschreibung
<code>import_root_directory</code>	<code>str</code>	Ordnerpfad für den Import

```
safety_admin.import_config(import_root_directory = "C:\\ws\\exported\\SafetyAdministration")
```

2.31. PLC Data - SoftwareUnit

Die Klasse stellt eine Software Unit in TIA Portal dar. Sie bietet Methoden zum Abrufen von Informationen über die Software Unit, zum Exportieren der Software Unit und zum Zugriff auf verschiedene Unterkomponenten der Software Unit, wie PLC-Variablentabellen, Programmbausteine, Systembausteine, Benutzerdatentypen und externe Quellen.

2.31.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = software_unit.get_name()
```

2.31.2. compile()

Übersetzen der Software Unit

```
software_unit.compile()
```

2.31.3. export_configuration(export_file_path: str)

Exportieren der Konfiguration der Software Unit

Parameter	Typ	Beschreibung
export_file_path	str	Vollständiger Ordnerpfad für den Export

```
software_unit.export_configuration(export_file_path = "C:\\ws\\export\\config.xml")
```

2.31.4. import_configuration(import_file_path: str)

Importieren der Software Unit Konfiguration

Parameter	Typ	Beschreibung
import_file_path	str	Vollständiger Ordnerpfad der Konfiguration

```
software_unit.import_configuration(import_file_path = "C:\\ws\\export\\config.xml")
```

2.31.5. get_plc_tag_tables()

Abrufen der Liste der PLC-Variablentabellen

Returns → `List[PlcTagTable]` Liste der PLC-Variablentabellen der Software Unit

```
plc_tag_tables = software_unit.get_plc_tag_tables()
```

2.31.6. get_program_blocks()

Abrufen der Liste der Programmbausteine

Returns → `List[ProgramBlock]` Liste der Programmbausteine der Software Unit

```
blocks = software_unit.get_program_blocks()
```

2.31.7. get_system_blocks()

Abrufen der Liste der Systembausteine

Returns → `List[SystemBlock]` Liste der Systembausteine der Software Unit

```
blocks = software_unit.get_system_blocks()
```

2.31.8. get_user_data_types()

Abrufen der Liste der PLC-Datentypen

Returns → `List[UserDataTypes]` Liste der PLC-Datentypen der Software Unit

```
udts = software_unit.get_user_data_types()
```

2.31.9. get_external_sources()

Abrufen der Liste der externen Quellen

Returns → `List[ExternalSource]` Liste der externen Quellen der Software Unit

```
ext_sources = software_unit.get_external_sources()
```

2.31.10. get_named_value_types()

Abrufen der Liste der Named value-Datentypen

Returns → `List[NamedValueType]` Liste der Named value-Datentypen der Software Unit

```
nvts = software_unit.get_named_value_types()
```

2.31.11. import_blocks(import_root_directory: str)

Importieren der Programmbausteine aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners

```
software_unit.import_blocks(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Program blocks")
```

2.31.12. import_plc_tags(import_root_directory: str)

Importieren der Variablen tabellen aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners

```
software_unit.import_plc_tags(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC tags")
```

2.31.13. import_data_types(import_root_directory: str)

Importieren der PLC-Datentypen aus einem Verzeichnis in die PLC

Parameter	Typ	Beschreibung
import_root_directory	str	Verzeichnis des Importordners

```
software_unit.import_data_types(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC data types")
```

2.31.14. export_cross_references(target_directory_path: str, filter: int)

Exportieren von Querverweisen des Software Unit Objekts Filter → Integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
software_unit.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.31.15. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.31.16. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.31.17. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.31.18. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.31.19. get_identifier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifizier()
```

2.32. PLC Data - SystemBlock

Die Klasse stellt einen Systembaustein in TIA Portal dar. Sie bietet verschiedene Methoden zur Verwaltung und Steuerung von Systembausteinen.

2.32.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.32.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.32.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.32.4. compile()

Übersetzen des Systembausteins

```
system_block.compile()
```

2.32.5. is_consistent()

Prüfen der Konsistenz des Systembausteins

Returns → `bool` Wahr, wenn konsistent

```
value = system_block.is_consistent()
```

2.32.6. export_cross_references(target_directory_path: str, filter: int)

Exportieren der Querverweise des Systembausteins

Filter → Integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
system_block.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.32.7. show_in_editor()

Öffnen des PLC-Objekts im Editor

```
plc_object.show_in_editor()
```

2.32.8. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.32.9. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → List[str] Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.32.10. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → str Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.32.11. get_path()

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → str Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.32.12. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → int Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)

- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.32.13. get_supported_export_format()

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.32.14. get_fingerprints()

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.32.15. get_identifizier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifizier()
```


2.33. PLC Data - TechnologyObject

Die Klasse repräsentiert ein Technologieobjekt in TIA Portal. Sie bietet verschiedene Methoden zur Verwaltung und Steuerung von Technologieobjekten.

2.33.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.33.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.33.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf `True` gesetzt ist, werden alle Group Names hierarchisch mit Foldern dargestellt.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.33.4. compile()

Übersetzen des Technologieobjekts

```
to.compile()
```

2.33.5. is_consistent()

Prüfen der Konsistenz des Technologieobjekts

Returns → `bool` True, wenn konsistent

```
value = to.is_consistent()
```

2.33.6. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.33.7. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.33.8. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → `str` Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.33.9. get_path()

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → `str` Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.33.10. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.33.11. get_supported_export_format()

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.33.12. get_fingerprints()

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.33.13. `get_identifier()`

Abrufen des Identifiers des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.34. PLC Data - UserConstant

Die Klasse stellt eine Benutzerkonstante in TIA Portal dar. Sie bietet Methoden, um Informationen über die Benutzerkonstante zu erhalten, Eigenschaften abzurufen und die Konstante zu exportieren.

2.34.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.34.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.34.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.34.4. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.34.5. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.34.6. **set_property(name:str, value: str)**

Setzen der Eigenschaft des Objekts

Returns → **int** Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.34.7. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → **List[str]** Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.34.8. **get_identifier()**

Abrufen des Identifier des Objekts

Returns → **str** Identifier als String

```
property_value = tiap_object.get_identifier()
```

2.35. PLC Data - UserData Type

Die Klasse repräsentiert einen PLC-Datentyp in TIA Portal. Sie bietet Methoden zur Verwaltung und Steuerung von PLC-Datentypen, zum Exportieren des Datentyps, zum Abrufen von Datentypinformationen und zur Überprüfung von Querverweisen.

2.35.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.35.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.35.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.35.4. compile()

Übersetzen des PLC-Datentyps

```
udt.compile()
```

2.35.5. is_consistent()

Prüfen der Konsistenz des PLC-Datentyps

Returns → `bool` True, wenn konsistent

```
value = udt.is_consistent()
```

2.35.6. export_cross_references(target_directory_path: str, filter: int)

Exportieren der Querverweise des PLC-Datentyps

filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
udt.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences", filter = 2)
```

2.35.7. get_type_version_guid()

Abrufen der GUID der Typversion

Returns → str GUID

```
guid = plc_object.get_type_version_guid()
```

2.35.8. get_type_guid()

Abrufen der GUID des Typs

Returns → str GUID

```
guid = plc_object.get_type_guid()
```

2.35.9. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.35.10. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → List[str] Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.35.11. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → str Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.35.12. get_path()

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → str Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.35.13. **set_property(name:str, value: str)**

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.35.14. **get_supported_export_format()**

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.35.15. **get_fingerprints()**

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.35.16. **is_library_type()**

Überprüfen ob das Objekt ein Bibliotheks-Typ ist

Returns → `bool` True, wenn es ein Bibliotheks-Typ ist

```
value = plcdata.is_library_type()
```

2.35.17. **get_identifier()**

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_identifier()
```


2.36. PLC Data - WatchTable

Die Klasse stellt eine Beobachtungstabelle in TIA Portal dar. Sie bietet Methoden zum Abrufen von Informationen über die Beobachtungstabelle, zum Abrufen von Eigenschaften und zum Exportieren der Tabelle.

2.36.1. get_name()

Abrufen des Namens des Objekts

Returns → `str` Name des Objekts

```
name = plc_object.get_name()
```

2.36.2. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.36.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Exportieren des PLC-Objekts

Wenn `keep_folder_structure` auf True gesetzt ist, werden alle Gruppennamen hierarchisch mit Ordnern exportiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export
export_options	Enums.ExportOptions	Optional: Exportoptionen, Standard ist ExportOptions.None
export_format	Enums.ExportFormats	Optional: Exportformat, Standard ist SimaticML
keep_folder_structure	bool	True: Alle Gruppennamen werden hierarchisch mit Ordnern exportiert

```
plc_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.36.4. is_consistent()

Prüfen der Konsistenz der Beobachtungstabelle

Returns → `bool` True, wenn konsistent

```
value = watch_table.is_consistent()
```

2.36.5. show_in_editor()

Öffnen des PLC-Objekts im Editor

```
plc_object.show_in_editor()
```

2.36.6. delete()

Löschen des PLC-Objekts

```
tiap_object.delete()
```

2.36.7. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.36.8. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.36.9. get_supported_export_format()

Abrufen der unterstützten Export-Formate

Returns → `List[str]` Unterstütztes Format

```
supported_formats = plc_object.get_supported_export_format()
```

2.36.10. get_path_full()

Abrufen des Ordnerpfads eines Objekts bis zum Projekt-Objekt

Returns → `str` Vollständiger Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.36.11. get_path()

Abrufen des Ordnerpfads eines Objekts bis zum relevanten System-Objekt

Returns → `str` Ordnerpfad

```
group_path = plcdata.get_path_full()
```

2.36.12. get_fingerprints()

Abrufen der Fingerprint-Daten des Objekts

Returns → `List[str]` Fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.36.13. `get_identifizier()`

Abrufen des Identifizier des Objekts

Returns → `str` Identifizier als String

```
property_value = tiap_object.get_identifizier()
```

2.37. Project - Project

Die Klasse repräsentiert ein TIA Portal Projekt und bietet Funktionen für allgemeine Projektoperationen, wie Speichern, Schließen, Aktivieren/Deaktivieren der Simulationsunterstützung, Archivieren, Verwalten von Hardware und mehr.

2.37.1. get_portal()

Abrufen der TIA Portal-Instanz aus dem aktuellen Projekt

```
portal = project.get_portal()
```

2.37.2. save()

Speichern des TIA Portal Projekts

```
project.save()
```

2.37.3. close()

Schließen des TIA Portal Projekts ohne ausstehende Änderungen zu speichern

WARNUNG: Alle ungespeicherten Änderungen gehen unwiderruflich verloren

```
project.close()
```

2.37.4. set_simulation_support(value: bool)

Einstellen der Simulationsunterstützung für das TIA Portal Projekt

Parameter	Typ	Beschreibung
value	bool	Setzt den Wert der Simulationsunterstützung

```
project.set_simulation_support(value = True)
```

2.37.5. archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)

Archivieren des TIA Portal Projekts

Returns → **str** Vollständiger Pfad des neuen Archivs

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad, in dem das Archiv abgelegt werden soll
archive_name	str	Name der Archivdatei
delete_existing_archive	bool	Legt fest, ob die Ausgabedatei zuerst gelöscht werden soll

```
archive_fullpath = project.archive(target_directory_path = "C:\\ws\\newfolder", archive_name = "testproj", delete_existing_archive = True)
```

2.37.6. save_as(target_directory_path: str, project_name: str)

Speichern des TIA Portal Projekts unter einem anderen Pfad und Namen

Returns → **str** Vollständiger Pfad des gespeicherten Projekts

Parameter	Typ	Beschreibung
target_directory_path	str	Pfad des Ordners, in dem das Projekt gespeichert werden soll

Parameter	Typ	Beschreibung
project_name	str	Name des Projekts

```
project_file_path = project.save_as(target_directory_path = "C:\\ws\\newfolder", project_name = "testproj")
```

2.37.7. export_cax_data(export_file_path: str, log_file_path: str)

Exportieren der CAX-Daten des TIA Portal Projekts

Returns → `bool` Status des Exports

Parameter	Typ	Beschreibung
export_file_path	str	Vollständiger Pfad der Exportdatei
log_file_path	str	Vollständiger Pfad der Protokolldatei

```
result = project.export_cax_data(export_file_path = "C:\\ws\\exportfolder\\exportCAX.aml", log_file_path = "C:\\ws\\exportfolder\\exportCAX.log")
```

2.37.8. import_cax_data(import_file_path: str, log_file_path: str)

Importieren der CAX-Daten in das TIA Portal Projekt

Returns → `bool` Status des Imports

Parameter	Typ	Beschreibung
import_file_path	str	Vollständiger Pfad der Importdatei
log_file_path	str	Vollständiger Pfad der Protokolldatei

```
result = project.import_cax_data(import_file_path = "C:\\ws\\importfolder\\importCAX.aml", log_file_path = "C:\\ws\\importfolder\\importCAX.log")
```

2.37.9. open_topology_editor()

Öffnen des Topologie-Editor in TIA Portal für das Projekt

```
project.open_topology_editor()
```

2.37.10. open_network_editor()

Öffnen des Netzwerkeditor in TIA Portal für das Projekt

```
project.open_network_editor()
```

2.37.11. get_plcs()

Abrufen der Liste der PLCs im TIA Portal Projekt

Returns → `List[Plc]` Alle PLC's als Liste

```
plcs = project.get_plcs()
for plc in plcs:
    ...
```

2.37.12. get_hmis()

Abrufen der Liste der HMIs im TIA Portal Projekt

Returns → `List[Hmi]` Alle HMIs als Liste

```
hmis = project.get_hmis()
for hmi in hmis:
...
```

2.37.13. get_application_tests()

Abrufen der Liste der Applikationstests der Test Suite im TIA Portal Projekt

Test Suite ist erforderlich.

Returns → `List[ApplicationTest]` Alle Applikationstests als Liste

```
tests = project.get_application_tests()
```

2.37.14. get_system_tests()

Abrufen der Liste der Systemtests der Test Suite im TIA Portal Projekt

Test Suite ist erforderlich.

Returns → `List[SystemTest]` Alle Systemtests als Liste

```
tests = project.get_system_tests()
```

2.37.15. get_rule_sets()

Abrufen der Liste der Styleguide-Regelsätze der Test Suite im TIA Portal-Projekt

Test Suite ist erforderlich.

Returns → `List[RuleSet]` Alle Regelsätze als Liste

```
rule_sets = project.get_rule_sets()
```

2.37.16. get_project_library()

Abrufen der Projektbibliothek

Returns → `ProjectLibrary` Projektbibliothek

```
projekt_lib = project.get_project_library()
```

2.37.17. web_block_generate()

Generieren von Web-Block-DBs entsprechend den vom Benutzer definierten Seiten

Webserver muss aktiviert sein.

```
project.web_block_generate()
```

2.37.18. upgrade_hardware(full_upgrade: bool)

Aktualisieren der Geräte auf die neueste Bestellnummer und Firmware-Version

Wenn `full_upgrade` auf True gesetzt ist, werden alle Geräte auf die neueste verfügbare Bestellnummer (Gerätetyp) und Firmware umgestellt:

von CPU1511F 6ES7 511-1FK00-0AB0 V1.7 auf 6ES7 511-1FK00-0AB0 V1.8.

Mit vollständigem Upgrade: von CPU1511F 6ES7 511-1FK00-0AB0 V1.7 auf 6ES7 511-1FL03-0AB0 V3.0.

Parameter	Typ	Beschreibung
full_upgrade	bool	Legt fest, ob ein vollständiges Upgrade durchgeführt werden soll (neueste Bestellnummer und Firmware-Version)

```
project.upgrade_hardware(full_upgrade = True)
```

2.37.19. sivarc_generate()

Starten der SiVArc-Generierung

SiVArc muss installiert und lizenziert sein.

```
project.sivarc_generate()
```

2.37.20. update_module_description()

Aktualisieren der Modulbeschreibung aller Geräte im Projekt

```
project.update_module_description()
```

2.37.21. set_virtual_plc_support(value: bool)

Einstellen der virtuellen PLC-Unterstützung im Projekt

Nur in V19 oder höher verfügbar

Parameter	Typ	Beschreibung
value	bool	Den Value des Virtual Support setzen

```
project.set_virtual_plc_support(value = "True")
```

2.37.22. import_umac_config(import_file_path: str)

Importieren der UMAC- und UMC-Konfiguration in das TIA Portal Projekt

Returns → `int` Statuswert des Import

Parameter	Typ	Beschreibung
import_file_path	str	Vollständiger Pfad der Importdatei
secret	str	Vollständiger Pfad der Importdatei
secret_env_name	str	Name der Umgebungsvariablen, in der das Geheimnis gespeichert wird (optional, wenn Passwörter verschlüsselt)

```
project.import_umac_config(import_file_path = "C:\\ws\\importfolder\\importUMAC.json",
secret_env_name = "MYSECRETENV")
```

2.37.23. export_umac_config(export_file_path: str)

Exportieren der UMAC- und UMC-Konfiguration aus dem TIA Portal Projekt

Wenn die Exportdatei bereits existiert, wird sie überschrieben.

Parameter	Typ	Beschreibung
export_file_path	str	Vollständiger Pfad der Exportdatei

```
project.export_umac_config(export_file_path = "C:\\ws\\exportfolder\\exportUMAC.json")
```

2.37.24. import_password_policy(import_file_path: str)

Importieren von Kennwortrichtlinien in das TIA Portal Projekt

Parameter	Typ	Beschreibung
import_file_path	str	Vollständiger Pfad der Importdatei

```
project.import_password_policy(import_file_path = "C:\\ws\\importfolder\\PWPolicy.json")
```

2.37.25. export_password_policy(export_file_path: str)

Exportieren von Kennwortrichtlinien aus dem TIA Portal Projekt

Wenn die Exportdatei bereits existiert, wird sie überschrieben.

Parameter	Typ	Beschreibung
export_file_path	str	Vollständiger Pfad der Importdatei

```
project.export_password_policy(export_file_path = "C:\\ws\\exportfolder\\exportPWPolicy.json")
```

2.37.26. delete()

Löschen des TIA Portal Projekts

```
project.delete()
```

2.37.27. start_transaction(undo_text: str, dialog_text: str)

Starten des exklusiven Zugriffs und einer neuen Transaktion

Parameter	Typ	Beschreibung
undo_text	str	Text auf der Schaltfläche Rückgängig machen
dialog_text	str	Text im Dialog während der Transaktion

```
project.start_transaction(undo_text = "MyUndoDescription", dialog_text = "MyExclusiveAccess")
```

2.37.28. end_transaction(rollback: Optional[bool] = None)

Beenden der Transaktion und exklusiven Zugriffs

Parameter	Typ	Beschreibung
Rollback	bool	Legt fest, ob die Änderungen in der Transaktion rückgängig gemacht werden sollen (Standardwert: false)

```
project.end_transaction()
```

2.37.29. update_transaction(dialog_text: str)

Aktualisieren der Transaktion des laufenden exklusiven Zugriffs

Parameter	Typ	Beschreibung
dialog_text	str	Text im Dialog während der Transaktion

```
project.update_transaction(dialog_text = "MyNewExclusiveAccess")
```

2.37.30. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → **str** Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.37.31. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.37.32. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.37.33. is_session()

Prüfen ob das aktuelle Projekt eine Lokale Session ist

Returns → `bool` True, wenn Lokale Session

```
is_session = project.is_session()
```

2.37.34. commit_and_close(commit_message: str)

Kommentieren der Änderungen und Schließen des Projekts

Returns → `int` Laufende Nummer des Commits

Parameter	Typ	Beschreibung
commit_message	str	Commit Kommentar

```
rev_number = project.commit_and_close(commit_message = "MyCommitMessage")
```

2.37.35. get_idenfier()

Abrufen des Identifier des Objekts

Returns → `str` Identifier als String

```
property_value = tiap_object.get_idenfier()
```

2.38. Project - ProjectServer

Die Klasse stellt einen TIA Project-Server im TIA Portal dar.

2.38.1. get_host()

Abrufen des Hosts des TIA Project-Servers

Returns → `str` Host

```
host = server_project.get_host()
```

2.38.2. get_port()

Abrufen des Ports des TIA Project-Servers

Returns → `int` Port-Nummer

```
port = server_project.get_port()
```

2.38.3. get_server_name()

Abrufen des Servernamens des TIA Project-Servers

Returns → `str` Servername

```
name = server_project.get_server_name()
```

2.38.4. print_info()

Anzeigen von Informationen über den TIA Project-Server

```
server_project.print_info()
```

2.38.5. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.38.6. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Eigenschaften

```
properties = tiap_object.get_properties()
```

2.38.7. set_property(name:str, value: str)

Setzen der Eigenschaft des Objekts

Returns → `int` Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“

- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.38.8. delete()

Löschen des Objekts

```
tiap_object.delete()
```

2.38.9. add_project(project_path: str)

Hinzufügen des Projekts zum Project-Server

Parameter	Typ	Beschreibung
project_path	str	Pfad der Projektdatei

```
project_server.add_project(project_path = "D:\tia_sessions\MyTestSession\MyTestSession.als")
```

2.38.10. create_local_session(server_project_path: str, session_directory_path: str, session_name: str, exclusive: Optional[bool] = None)

Erstellen einer Lokalen Session

Warnung: Project-Server Gruppen werden ab API V20 unterstützt - davor werden nur ungruppierte Server Projekte unterstützt und können abgerufen werden.

Returns → **str** Pfad zur Lokalen Session des TIA Portal Projekts

Parameter	Typ	Beschreibung
server_project_path	str	Pfad des Projekts auf dem Projekt-Server
session_directory_path	str	Lokaler Dateipfad zum Speichern der Lokalen Session
session_name	str	Name der Session
exclusive	bool	Default: true Öffnet exklusive Session

```
local_session_project_path = project_server.create_local_session(server_project_path = "myServerGroup/MyServerProject", session_directory_path = "C:\\ws\\sessionDir", session_name = "myLocalSession", exclusive = False)
```

2.38.11. get_server_project_paths(group: Optional[str] = None)

Abrufen aller Projekt-Pfade vom ausgewählten TIA Project-Server

Returns → **List[str]** Pfade der Projekte

Parameter	Typ	Beschreibung
group	str	Optional: Gruppenname

```
project_names = project_server.get_server_project_paths(group = "MyGroup")
```

2.38.12. delete_local_session(session_file_path: str, server_project_path: str)

Löschen der Lokalen Session

Parameter	Typ	Beschreibung
session_file_path	str	Pfad zur Lokalen Session. Unterstützt .amc und auch .als Dateipfade
server_project_path	str	Pfad des Projekts auf dem Server

```
project_server.delete_local_session(session_file_path = "C:\\ws\\tia-  
sessions\\myproject\\myproject.amc" , server_project_path = "myGroup/serverproject")
```

2.39. Test Suite - ApplicationTest

Die Klasse stellt einen Test Suite Applikationstest im Kontext von TIA Portal dar.

Test Suite Advanced muss installiert und lizenziert sein.

2.39.1. get_name()

Abrufen des Namens des Applikationstest

Returns → `str` Name des Applikationstest

```
name = app_test.get_name()
```

2.39.2. get_property(name: str)

Abrufen der Eigenschaft des Test Suite Applikationstests

Eigenschaften, die keine Strings sind, werden wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Eigenschaft des Applikationstests

```
property_value = app_test.get_property(name = "Property")
```

2.39.3. export(target_directory_path: str)

Exportieren des Test Suite Anwendungstest

Der Ordner "Application tests" wird automatisch im `target_directory_path` erstellt, falls er noch nicht existiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export

```
app_test.export(target_directory_path = "C:\\ws\\export")
```

2.39.4. set_scope(plc_name: str, instance_name: Optional[str] = None, execution_mode: Optional[int] = None)

Festlegen des Scopes des Test Suite Applikationstests

Parameter	Typ	Beschreibung
plc_name	str	Name der zu verwendenden PLC
instance_name	str	PLCSIM Instanzname (nur V19 und höher)
execution_mode	integer (enum)	(nur V19 und höher) [1: 'SystemManagedPLCSIMInstance', 2: 'ExternallyManagedPLCSIMInstance']

```
app_test.set_scope(plc_name = "PLC_1")
```

2.40. Test Suite - RuleSet

Die Klasse repräsentiert einen Test Suite Styleguide Regelsatz im Kontext von TIA Portal.

Test Suite Advanced muss installiert und lizenziert sein.

2.40.1. get_name()

Abrufen des Namens des Styleguide Regelsatzes

Returns → `str` Name des Regelsatzes

```
name = rule.get_name()
```

2.40.2. get_property(name: str)

Abrufen der Eigenschaft des Styleguide Regelsatzes

Eigenschaften, die keine Strings sind, werden wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Eigenschaft des Regelsatzes

```
property_value = rule.get_property(name = "Property")
```

2.40.3. export(target_directory_path: str)

Exportieren des Styleguide Regelsatzes

Der Ordner "Style guide" wird automatisch im `target_directory_path` erstellt, falls er noch nicht existiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export

```
rule.export(target_directory_path = "C:\\ws\\export")
```

2.41. Test Suite - SystemTest

Die Klasse stellt einen Test Suite Systemtest im Kontext von TIA Portal dar.

Test Suite Advanced muss installiert und lizenziert sein.

2.41.1. get_name()

Abrufen des Namens des Test Suite Systemtests

Returns → `str` Name des Systemtests

```
name = sys_test.get_name()
```

2.41.2. export(target_directory_path: str)

Exportieren des Test Suite Systemtests

Der Ordner "System tests" wird automatisch im `target_directory_path` erstellt, falls er noch nicht existiert.

Parameter	Typ	Beschreibung
target_directory_path	str	Ordnerpfad für den Export

```
sys_test.export(target_directory_path = "C:\\ws\\export")
```

2.41.3. set_scope(opcua_server_address: str, opcua_server_interface_type: int, opcua_server_interface_folder_path: Optional[str] = None)

Festlegen des Scopes des Test Suite Systemtests

Parameter	Typ	Beschreibung
opcua_server_address	str	OPC UA Server Adresse, z.B. <code>opc.tcp://server.port/path</code>
opcua_server_interface_type	integer (enum)	[1: 'UserDefined', 2: 'StandardSIMATIC']
opcua_server_interface_folder_path	str	Ordnerpfad zu Schnittstellendateien

```
sys_test.set_scope(opcua_server_address = "opc.tcp://server.port/path",  
opcua_server_interface_type = 1)
```

2.41.4. get_property(name: str)

Abrufen der Eigenschaften des Objekts

Eigenschaften, die keine Strings sind, werden, wenn möglich, in Strings umgewandelt.

Returns → `str` Wert der Eigenschaft als String

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.41.5. get_properties()

Abrufen aller Eigenschaften des Objekts

Returns → `List[str]` Namen der Attribute

```
properties = tiap_object.get_properties()
```

2.41.6. **set_property(name:str, value: str)**

Setzen der Eigenschaft des Objekts

Returns → **int** Rückgabewert des Ergebnisses

Setzt eine Eigenschaft des Objekts, indem der gegebene Wert zugewiesen wird, sofern er im korrekten Format vorliegt.

- **Boolean:** „True“ oder „False“
- **Integer:** Ganze Zahlen (z.B. „42“)
- **Double:** Dezimalzahlen mit einem Punkt (z.B. „3.14“)
- **Enum:** Entweder der Enumindex (z.B. „0“, „1“, „2“) oder der entsprechende Wert (z.B. „OptionA“, „OptionB“).

Parameter	Typ	Beschreibung
name	str	Name der Eigenschaft des Objekts
value	str	Wert der Eigenschaft, z.B. „1“, „True“, „MyNewString“

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.41.7. **get_identifizier()**

Abrufen des Identifizier des Objekts

Returns → **str** Identifizier als String

```
property_value = tiap_object.get_identifizier()
```


3. Anhang

3.1. Service und Support

SiePortal

Die integrierte Plattform für Produktauswahl, Einkauf und Support – und Verbindung von Industry Mall und Online Support. Die neue Startseite, ersetzt die bisherigen Startseiten der Industry Mall sowie des Online Support Portals (SIOS) und fasst diese zusammen.

- **Produkte & Services**
Unter Produkte & Services finden Sie alle unsere Angebote, die bisher im Mall Katalog verfügbar waren.
- **Support**
Im Bereich Support finden Sie alle Informationen, die für die Lösung technischer Probleme mit unseren Produkten hilfreich sind.
- **mySieportal**
mySiePortal ist Ihr persönlicher Bereich, der Funktionen, wie z.B. die Warenkorbverwaltung oder die Bestellübersicht anzeigt. Den vollen Funktionsumfang sehen Sie hier erst nach erfolgreichem Login.

Das SiePortal rufen Sie über diese Adresse auf: sieportal.siemens.com

Technical Support

Der Technical Support von Siemens Industry unterstützt Sie schnell und kompetent bei allen technischen Anfragen mit einer Vielzahl maßgeschneiderter Angebote – von der Basisunterstützung bis hin zu individuellen Supportverträgen.

Anfragen an den Technical Support stellen Sie per Web-Formular:

support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

Mit unseren weltweit verfügbaren Trainings für unsere Produkte und Lösungen unterstützen wir Sie praxisnah, mit innovativen Lernmethoden und mit einem kundenspezifisch abgestimmten Konzept.

Mehr zu den angebotenen Trainings und Kursen sowie deren Standorte und Termine erfahren Sie unter:

siemens.com/sitrain

Industry Online Support App

Mit der App "Industry Online Support" erhalten Sie auch unterwegs die optimale Unterstützung.

Die App ist für iOS und Android verfügbar:



3.2. Links und Literatur

Tabelle 4-1

Nr.	Thema
\1	SiePortal https://sieportal.siemens.com/de-ww
\2	Link auf die Beitragsseite des Anwendungsbeispiels https://support.industry.siemens.com/cs/ww/de/view/109742322

3.3. Änderungsdokumentation

Tabelle 4-2

Version	Datum	Modifikationen
V1.2.1	12/2025	fix: Hotfix für TIA Portal Openness V21
V1.2.0	12/2025	- feat: Unterstützung TIA Portal Openness V21 hinzugefügt - feat: Unterstützung PLC Download und Go Online hinzugefügt
V1.1.0	09/2025	- feat!: Verschieben von Projekt-Sessions zum ProjectServer-Objekt - feat: Einfügen von Funktionalität für HMI (Bilder, Variablen, Skripte, Verbindungen, Zyklen, Alarme, Text- und Grafiklisten, ...) - feat: Unterstützung für Export und Import im Format SIMATIC SD (Source Documents) hinzugefügt - feat: Unterstützung zum Setzen von Objekt-Eigenschaften hinzugefügt - feat: Unterstützung für Bibliotheks-Anwendungsfälle hinzugefügt (Ordner verwalten, Typen finden, Instanzieren, Aktualisieren, Bereinigen, Bearbeiten, Freigeben, Verwerfen, Exportieren) - feat: Unterstützung für PLC-Objekte hinzugefügt (Know-How-Schutz, Fingerprints) - feat: Funktionalität zum Erstellen neuer Projekte und Bibliotheken hinzugefügt - feat: Unterstützung für Project-Server-Gruppen hinzugefügt - feat: Unterstützung für Objekt-Identifikatoren hinzugefügt - feat: Beispielskripte aktualisiert - fix: Verbesserungen anhand von Nutzer-Rückmeldungen
V1.0.7	10/2024	Erstveröffentlichung